



**UNIVERSIDAD AUTÓNOMA DE
QUERÉTARO**

FACULTAD DE INGENIERÍA

**Predicción de la Inflación en México:
Un Estudio Comparativo entre Modelos Clásicos, Árboles
de Decisión y Redes Neuronales**

TESIS

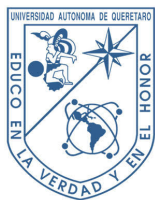
**PARA OBTENER EL TÍTULO DE
LICENCIATURA EN MATEMÁTICAS APLICADAS**

PRESENTA

Norberto Peña Rodríguez

DIRIGIDA POR

Dr. Víctor Antonio Aguilar Arteaga



UNIVERSIDAD AUTÓNOMA DE QUERÉTARO

FACULTAD DE INGENIERÍA

**Predicción de la Inflación en México:
Un Estudio Comparativo entre Modelos Clásicos, Árboles
de Decisión y Redes Neuronales**

TESIS

**PARA OBTENER EL TÍTULO DE
LICENCIATURA EN MATEMÁTICAS APLICADAS**

PRESENTA

Norberto Peña Rodríguez

DIRIGIDA POR

Dr. Víctor Antonio Aguilar Arteaga

Dr. Víctor Antonio Aguilar Arteaga

Presidente

M en C Iván González García

Secretario

Dr. Francisco Gerardo Jiménez López

Vocal

M.C. Juan Pablo Ramirez Vazquez

Suplente

Abstract

Inflation constitutes one of the most complex economic phenomena to model, due to its dynamic and nonlinear nature, as well as the presence of structural and stochastic components that evolve over time. In this research, a forecasting system for inflation in Mexico is developed, integrating classical methodologies, machine learning algorithms, and deep learning models, with the objective of comparing their performance, stability, and generalization capability under different validation schemes.

The study proposes a unified framework that automates training, validation, and forecasting through Bayesian optimization and cross-validation, applicable to different forecasting horizons. Models ranging from Holt–Winters triple exponential smoothing and XGBoost to advanced architectures such as RNN, LSTM, DeepAR, Transformer, and D³VAE are implemented, exploring how each approach captures the seasonality, trend, and nonlinearity inherent in the inflationary process.

The results show that, although deep neural networks can fit the observed inflation dynamics with high accuracy, their performance is not consistent across all scenarios, exhibiting higher variance under cross-validation, particularly when incorporating temporal exogenous variables and signals derived from the Fourier Transform. In contrast, attention-based models and probabilistic generative approaches, particularly Transformer and D³VAE, exhibit a better balance between accuracy and stability, achieving the best performance in medium- and long-term forecasting horizons.

Finally, the study demonstrates that simple statistical models, such as the historical average and the naive forecast based on the last observed value, constitute surprisingly competitive baselines due to their low variance and robustness. These findings underscore that the use of complex models does not inherently guarantee superior results, and highlight the importance of combining statistical techniques and deep learning within a systematic, explainable, and scalable framework for the analysis and monitoring of inflation at the national level.

Keywords: Inflation in Mexico; time series forecasting; machine learning; deep learning; Bayesian optimization.

Resumen

La inflación constituye uno de los fenómenos económicos más complejos de modelar, debido a su naturaleza dinámica, no lineal y a la presencia de componentes estructurales y estocásticos que varían en el tiempo. En esta investigación se desarrolla un sistema de predicción para la inflación en México, integrando metodologías clásicas, algoritmos de aprendizaje automático y modelos de aprendizaje profundo, con el objetivo de comparar su desempeño, estabilidad y capacidad de generalización bajo distintos esquemas de validación.

El estudio propone un marco unificado que automatiza el entrenamiento, validación y pronóstico mediante optimización Bayesiana y validación cruzada, aplicable a distintos horizontes temporales. Se implementan modelos desde el suavizado exponencial triple de Holt–Winters y XGBoost, hasta arquitecturas avanzadas como RNN, LSTM, DeepAR, Transformer y D³VAE, explorando cómo cada enfoque captura la estacionalidad, tendencia y no linealidad inherentes al proceso inflacionario.

Los resultados muestran que, si bien las redes neuronales profundas pueden ajustarse con gran precisión a la dinámica observada de la inflación, su desempeño no es consistente en todos los escenarios, presentando una mayor varianza bajo validación cruzada, particularmente al incorporar variables exógenas temporales y señales derivadas de la Transformada Discreta de Fourier. En contraste, los modelos basados en atención y los enfoques generativos probabilísticos, en particular Transformer y D³VAE, exhiben un mejor balance entre precisión y estabilidad, logrando el mejor desempeño en horizontes de mediano y largo plazo.

Finalmente, el estudio evidencia que modelos estadísticos simples, como el promedio histórico y el pronóstico ingenuo basado en el último valor observado, constituyen líneas base sorprendentemente competitivas, debido a su baja varianza y robustez. Estos hallazgos subrayan que el uso de modelos complejos no garantiza mejores resultados, y resaltan la importancia de combinar técnicas estadísticas y de aprendizaje profundo dentro de un marco sistemático, explicable y escalable para el análisis y seguimiento de la inflación a nivel nacional.

Palabras clave: Inflación en México; pronóstico de series temporales; aprendizaje automático; aprendizaje profundo; optimización bayesiana.

Dedicatoria

A mis padres y a mi familia, porque antes de cualquier logro, modelo, resultado o título, existió su presencia. Porque en los momentos en los que dudé de mí, de mi camino o de mi capacidad para terminar lo que había comenzado, su amor siguió siendo una forma silenciosa de sostén. Esta tesis también les pertenece, porque mucho de lo que soy nació de su esfuerzo, de su paciencia y de su manera de acompañarme incluso cuando yo mismo no sabía explicar lo que estaba viviendo.

A mis amigos, especialmente a quienes permanecieron en las buenas y en las malas. A quienes me escucharon, me cuestionaron, me hicieron reír, me confrontaron y, sin darse cuenta, me recordaron que uno no se construye solo. En cada conversación profunda, en cada consejo, en cada momento compartido, hubo una parte de mí que aprendió a mirar la vida con más claridad.

A Castaño, por las ideas, las discusiones y las referencias compartidas; por recordarme que el conocimiento también se forma en el diálogo, en la curiosidad y en la disposición de mirar un problema desde otro lugar.

A Víctor, mi director de tesis, por abrirme la puerta a este tema, por orientarme con libertad y por permitirme desarrollar una investigación que terminó siendo mucho más que un requisito académico. Gracias por ayudarme a encontrar en la inflación, en los modelos y en los datos, una forma distinta de pensar el mundo.

Dedico también este trabajo a todas las personas que, aun sin permanecer para siempre, dejaron algo en mí. A quienes me inspiraron, a quienes me enseñaron que crecer también implica perder, cambiar, soltar y volver a empezar. Hay personas que pasan por la vida como estrellas fugaces, que iluminan nuestras vidas en un abrir y cerrar de ojos; otras, como una respuesta; y otras, simplemente como un recordatorio de lo lindo que es vivir y contribuir significativamente a este mundo. Todas ellas, de alguna forma, me trajeron hasta aquí.

Finalmente, dedico esta tesis a esa parte de mí que siguió buscando sentido en las matemáticas, en la ciencia, en el arte, en el trabajo, en el amor y en la posibilidad de ser mejor. Esta tesis es testimonio de ese camino.

Agradecimientos

Quiero expresar mi más sincero agradecimiento a mis padres y a mi familia, por su apoyo constante, comprensión y acompañamiento a lo largo de mi formación académica y personal. Su respaldo ha sido fundamental para la culminación de este trabajo. Agradezco también a mis amigos, quienes a través del diálogo, la reflexión y el cuestionamiento continuo han contribuido a ampliar mi forma de pensar y de entender el mundo. Un agradecimiento especial a Castaño, por la información, discusiones y referencias compartidas, las cuales resultaron valiosas durante el desarrollo de esta investigación. Asimismo, expreso mi gratitud a Víctor, mi director de tesis, por su apertura, orientación y por incentivar mi interés en el estudio de este tema, lo que derivó en un acercamiento más profundo al análisis económico y financiero. Finalmente, agradezco a todas las personas que, de una u otra forma, formaron parte de este proceso y contribuyeron a mi crecimiento personal y académico.

Abreviaturas y siglas

Banxico	Banco de México.
INEGI	Instituto Nacional de Estadística y Geografía.
IPC	Índice Nacional de Precios al Consumidor.
DFT	Transformada Discreta de Fourier.
KL	Divergencia de Kullback–Leibler.
MAE	Error absoluto medio.
RMSE	Raíz del error cuadrático medio.
MAPE	Error porcentual absoluto medio.
XGBoost	Extreme Gradient Boosting.
RNN	Red neuronal recurrente.
LSTM	Long Short-Term Memory.
DeepAR	Modelo autorregresivo profundo probabilístico.
VAE	Autoencoder variacional.
D³VAE	Modelo variacional basado en procesos de difusión para series temporales.



Figura 1: *

Autodeterminismo del Free Fire

Autor: Ignacio Chávez (2025).

Técnica: Óleo sobre lino

Medidas: 90cm x 90cm

Fuente: Cortesía del autor.

Índice

Abstract	i
Resumen	ii
Dedicatoria	iii
Agradecimientos	iv
Abreviaturas y siglas	v
Índice general	vii
Índice de cuadros	x
Índice de figuras	xi
Índice de imágenes	xii
1 Introducción	1
1.1 Planteamiento del problema	2
1.2 Justificación	2
1.3 Hipótesis	2
1.4 Objetivo general	2
1.5 Objetivos específicos	3
2 Antecedentes	3
3 Fundamentos Teóricos	4
3.1 Inflación	4
3.2 Series Temporales (conceptos base)	5
3.2.1 Serie temporal	5
3.2.2 Esperanza	6
3.2.3 Varianza	6
3.2.4 Autocovarianza	6
3.2.5 Autocorrelación	6
3.2.6 Estacionariedad	7
3.2.7 Descomposición mediante el filtro de Hodrick–Prescott . .	9
3.2.8 Transformaciones aplicadas	10
3.3 Embedding	11
3.4 Fundamentos probabilísticos	12
3.4.1 Operaciones básicas sobre distribuciones de probabilidad .	12
3.4.2 Logaritmo de una distribución normal	12
3.4.3 Divergencia de Kullback–Leibler	13
3.5 Transformación Discreta de Fourier (DFT)	13
3.6 Fundamentos estadísticos del aprendizaje	14

3.6.1	Formulación general del problema de aprendizaje	14
3.6.2	Error reducible e irreducible	15
3.6.3	Descomposición sesgo-varianza	15
3.6.4	Métricas de evaluación	15
3.6.5	Error de predicción y evaluación fuera de muestra	16
3.6.6	Linealidad y mínimos cuadrados	16
4	Espectro de Modelos de Pronóstico de Inflación: De Enfoques Simples a Arquitecturas Avanzadas de Aprendizaje Profundo	17
4.1	Naive	17
4.2	Promedio	17
4.3	Suavizado Exponencial Simple	17
4.4	Suavizado Exponencial Doble (Holt)	18
4.4.1	Nivel	18
4.4.2	Tendencia o Pendiente	18
4.5	Suavizado Exponencial Triple - Holt Winters	19
4.5.1	Estación	20
4.5.2	Longitud de una Estación	20
4.5.3	Componentes Estacionales	20
4.5.4	Adición vs Multiplicación	20
4.6	XGBoost	22
4.6.1	Aprendizaje de Árboles	23
4.6.2	Gradient Tree Boosting	23
4.6.3	Evaluación de la estructura de un árbol y la división del espacio	24
4.6.4	Interpretación de g_i , h_i y w_j	25
4.6.5	Shrinkage y Submuestreo de Columnas	25
4.6.6	Sistema de Diseño	26
4.6.7	Estructuras de datos optimizadas	26
4.6.8	Paralelización del proceso de construcción del árbol	26
4.6.9	Soporte para aprendizaje distribuido	27
4.6.10	Manejo de memoria y caché	27
4.6.11	Soporte para diferentes objetivos y métricas	27
4.7	Redes Neuronales Recurrentes (RNN)	28
4.8	Red Neuronal Recurrente con Memoria de Largo y Corto Plazo (LSTM)	33
4.9	DeepAr	36
4.10	Transformer	38
4.10.1	Codificador	38
4.10.2	Representación vectorial - Embeddings	39
4.10.3	Codificación Posicional	40
4.10.4	Mecanismo de Atención	41
4.10.5	Atención de Cabeza Múltiple - Multi-Head Attention	43
4.10.6	Conexiones Residuales	43
4.10.7	Capa de Normalización	43
4.10.8	Capa Feed-Forward por posición	44

4.10.9	Conexión Residual y Capa de Normalización	44
4.10.10	Sistema Completo - Codificador	45
4.10.11	Decodificador	45
4.10.12	Embeddings de entrada al decodificador	45
4.10.13	Atención enmascarada en el decodificador	45
4.10.14	Conexión Residual y Normalización	47
4.10.15	Atención cruzada (Encoder–Decoder Attention)	47
4.10.16	Capa Feed-Forward por posición	48
4.10.17	Predicción final y generación autoregresiva	48
4.11	D ³ VAE	49
4.11.1	Formulación del Problema	49
4.11.2	Preposición 1: Descomposición de señal y ruido	49
4.11.3	Proceso de Difusión Acoplada	50
4.11.4	Auto-codificador Variacional Bidireccional (BVAE)	51
4.11.5	Corrección de series generadas y Scaled Denoising Score Matching	52
4.11.6	Desentrelazando Z para Interpretabilidad	53
4.11.7	Entrenamiento y Predicción	55
5	Metodología	56
5.1	Conjunto de datos y frecuencia	56
5.2	Preprocesamiento y transformaciones	56
5.3	Ingeniería de variables	57
5.4	Modelos	57
5.5	Protocolos de validación	57
5.6	Optimización de hiperparámetros	58
5.7	Protocolo de pronóstico final	58
5.8	Controles de sobreajuste y linealidad	58
5.9	Diseño del experimento	58
5.10	Arquitectura del framework y reproducibilidad	59
6	Resultados y discusión	59
6.1	Diseño de evaluación	60
6.2	Escenarios de variables	60
6.3	Línea base vs. validación cruzada	60
6.4	Impacto de las variables exógenas	61
6.5	Ranking de modelos y estabilidad	61
6.6	Casos ilustrativos	62
6.7	Control de linealidad	63
6.8	Discusión frente a la literatura	63
6.9	Extensión HP y resultados pendientes	64
6.10	Hallazgos principales	64
7	Conclusiones	65
A	Código de optimización para RNN	66

Índice de tablas

1	Ejemplo de la lista ordenada de todos los tokens (palabras) introducidos al modelo	40
2	MAE (media $\pm$ sd) por protocolo y horizonte.	60
3	MAE promedio (CV, 12m) por modelo y escenario de variables. .	61
4	Matriz de resultados prevista para el benchmark HP. Los valores permanecen pendientes hasta ejecutar el protocolo completo. . .	64

Índice de figuras

1	*	vi
2	Comportamiento histórico de la inflación en México.	5
3	Función de autocorrelación (ACF) de la inflación en México.	7
4	Primera diferencia de la inflación mensual en México, utilizada para inducir estacionariedad.	8
5	Inflación mensual en México: serie original, tendencia estimada mediante el filtro HP y componente cíclico.	10
6	Representación conceptual del embedding: cada observación de la serie de inflación se proyecta a un espacio vectorial tridimensional.	11
7	Detección de Pronósticos Lineales	17
8	Ejemplo de un árbol de decisión proveniente de [10].	22
9	Estructura interna de una neurona recurrente, con funciones de activación ReLU.	28
10	Estructura interna de una neurona recurrente	33
11	Estructura de DeepAr, imagen de [24]	36
12	Arquitectura general del modelo Transformer, adaptado de Vaswani et al. (2017) [28].	39
13	Codificación posicional para la oración “ <i>Carolina come un omelet con Sur en un lindo lugar</i> ” con $d = 4$. Cada dimensión representa una componente del vector posicional asociado a cada palabra, y varía según su posición en la secuencia.	41
14	Estructura de DVAE, imagen de [15]	49
15	Valor real y pronóstico generado por el modelo D ³ VAE para una serie temporal sintética.	54
16	Comparación del MAE bajo validación cruzada (12 meses) para horizontes de 6 y 12 meses. Las barras representan el promedio y las líneas verticales la desviación estándar.	62
17	Ejemplo ilustrativo de pronóstico con LSTM bajo una configuración de hiperparámetros optimizada.	62
18	Ejemplo ilustrativo de pronóstico con LSTM incorporando suavizado exponencial para reducir ruido de alta frecuencia sin alterar la estructura general de la señal.	63

Índice de imágenes

Autodeterminismo del Free Fire, Ignacio Chávez (2025)	vi
---	----

1 Introducción

En un contexto económico caracterizado por alta volatilidad y constantes perturbaciones externas, anticipar la trayectoria de la inflación se ha convertido en una tarea fundamental para la toma de decisiones financieras, empresariales y gubernamentales. La predicción precisa de este indicador no solo permite estimar el poder adquisitivo y las tasas de interés, sino también planificar políticas públicas y estrategias corporativas en un entorno incierto.

Históricamente, el análisis inflacionario en México se ha basado en modelos econométricos lineales y técnicas de suavizado, que si bien han aportado interpretabilidad, presentan limitaciones para capturar las no linealidades y los rezagos temporales que caracterizan la dinámica inflacionaria. En respuesta a ello, el auge del aprendizaje automático y de las redes neuronales recurrentes ha abierto nuevas posibilidades para abordar la inflación como un proceso secuencial complejo, donde la información pasada y las variables exógenas interactúan de manera no trivial.

En esta investigación se desarrolla un sistema modular de predicción capaz de comparar distintos paradigmas: (1) modelos clásicos basados en series temporales como Holt-Winters, (2) modelos de aprendizaje automático como XGBoost, y (3) modelos neuronales profundos —RNN, LSTM, DeepAR, Transformer y D³VAE— que aprenden directamente la estructura temporal y estacional de los datos. Para garantizar una comparación justa y reproducible, cada modelo es entrenado mediante optimización Bayesiana y validación cruzada, implementadas en un orquestador diseñado específicamente para automatizar el proceso de entrenamiento y evaluación de cientos de configuraciones por modelo.

El conjunto de datos principal proviene del Índice Nacional de Precios al Consumidor (INPC) publicado por el INEGI, transformado a frecuencia semanal para capturar fluctuaciones de corto plazo sin perder la estructura de largo plazo. Además, se incorporan variables exógenas y codificaciones cíclicas derivadas de la fecha, junto con señales senoidales obtenidas mediante transformada de Fourier, con el objetivo de enriquecer la representación temporal de los modelos.

A través de este enfoque se busca no solo determinar qué arquitectura predice mejor la inflación mexicana, sino también entender por qué lo hace: cómo cada modelo procesa la información, qué tan sensibles son a las transformaciones aplicadas y en qué medida logran anticipar los patrones de estacionalidad y tendencia que definen la dinámica inflacionaria.

El valor central de este trabajo radica en su carácter comparativo, explicativo y aplicado. Al integrar metodologías de diferentes paradigmas bajo un mismo marco experimental, la tesis ofrece una visión integral del estado actual de la predicción inflacionaria y propone una metodología adaptable a otros contextos macroeconómicos, donde la precisión, la interpretabilidad y la automatización convergen como ejes fundamentales para la toma de decisiones basadas en datos.

El documento se organiza de la siguiente manera. El capítulo de antecedentes revisa los trabajos relacionados con el pronóstico de la inflación en México. Posteriormente, los fundamentos teóricos presentan los conceptos de series temporales, las transformaciones, el filtro de Hodrick–Prescott y las bases matemá-

ticas de los modelos considerados. El capítulo dedicado al espectro de modelos describe cada arquitectura; la metodología explica el conjunto de datos, la validación y el proceso de entrenamiento; finalmente, los resultados y la discusión contrastan el desempeño observado, antes de presentar las conclusiones y los anexos.

1.1 Planteamiento del problema

La inflación es un proceso dinámico en el que conviven variaciones persistentes de largo plazo y oscilaciones de menor duración. Cuando ambos comportamientos se modelan como una sola señal, una red neuronal debe aprender simultáneamente escalas temporales distintas, lo cual puede aumentar la varianza del entrenamiento y conducir a pronósticos triviales o poco estables. El problema central de esta investigación consiste, por tanto, en determinar qué modelos producen pronósticos precisos y estables de la inflación mexicana bajo un protocolo temporal comparable, y si separar explícitamente la tendencia y el ciclo permite estudiar el problema de una forma más justa y reproducible.

1.2 Justificación

Anticipar la inflación tiene relevancia social porque sus variaciones afectan el poder adquisitivo, la planeación familiar y las decisiones de ahorro y consumo. También posee relevancia económica y científica: instituciones públicas y privadas requieren estimaciones confiables, mientras que la comparación entre modelos permite estudiar hasta qué punto una arquitectura compleja aporta información adicional frente a líneas base simples. La construcción de un framework reproducible busca que las conclusiones no dependan de una sola fecha de corte ni de una librería particular, y que el procedimiento pueda extenderse a otros componentes del INPC.

1.3 Hipótesis

La descomposición de la inflación mediante el filtro de Hodrick–Prescott, seguida del modelado independiente de la tendencia y el componente cíclico, permite obtener pronósticos con menor error promedio y menor variabilidad temporal que el modelado directo de la serie, siempre que la descomposición y las transformaciones se ajusten únicamente con la información disponible antes de cada fecha de corte.

1.4 Objetivo general

Comparar el desempeño, la estabilidad y la capacidad de generalización de modelos clásicos, árboles de decisión y redes neuronales para pronosticar la inflación en México, incorporando un protocolo reproducible de validación temporal y una estrategia de descomposición en tendencia y ciclo.

1.5 Objetivos específicos

1. Construir un conjunto de datos semanal de la inflación mexicana a partir de la información publicada por el INEGI.
2. Implementar un framework común de entrenamiento, optimización, validación y pronóstico para los modelos considerados.
3. Evaluar el efecto de las transformaciones, las variables temporales y las representaciones espectrales sobre el desempeño fuera de muestra.
4. Implementar RNN, LSTM, DeepAR y Transformer directamente en PyTorch para reducir la dependencia del pipeline respecto a NeuralForecast.
5. Descomponer la serie mediante el filtro HP y pronosticar de manera independiente la tendencia τ_t y el ciclo c_t , evitando fuga de información.
6. Comparar los modelos mediante MAE promedio y dispersión bajo validación temporal para horizontes de seis y doce meses.

2 Antecedentes

Diversos estudios han analizado la inflación en México con diferentes enfoques. [2] compararon modelos basados en markup, la Curva de Phillips y modelos monetarios, encontrando que el modelo de sobre-costo tuvo el mejor desempeño predictivo para el período 1997-2001, con un RMSE de 0.038 y un MAE de 0.033, superando ampliamente a los demás enfoques. Estos resultados destacan la relevancia de considerar factores como los costos laborales y el tipo de cambio en la modelación de la inflación.

Por otro lado, el uso de modelos de series de tiempo para la predicción económica ha sido ampliamente estudiado en la literatura. En particular, Capistrán, Constandse y Ramos-Francia [5] analizaron la efectividad de modelos estacionales para pronosticar la inflación de corto plazo en México. En su estudio, evaluaron el desempeño de cuatro modelos estacionales utilizando datos de 16 índices del Índice Nacional de Precios al Consumidor (INPC), considerando tanto estacionalidad determinista como estocástica.

Los resultados indicaron que los modelos que asumen una estacionalidad y tendencia determinísticas producen los pronósticos más precisos. Además, los autores implementaron y compararon dos métodos de agregación de series de tiempo jerárquicas, encontrando que la combinación de modelos individuales mejora la precisión predictiva. De manera notable, sus pronósticos compitieron con los obtenidos en encuestas a especialistas en economía del sector privado elaboradas por el Banco de México.

Estos hallazgos resaltan la importancia de considerar la estacionalidad en la modelación de la inflación y sirven como base para el presente estudio, en el que se exploran enfoques complementarios para mejorar la predicción de la inflación en México mediante técnicas de aprendizaje automático.

El uso de redes neuronales para la predicción de la inflación en México ha sido explorado en diversos estudios. Pedroza Robles [23] analizó la capacidad predictiva de las redes neuronales en series de tiempo no lineales, mostrando su eficacia como técnica de estimación no paramétrica. Su estudio destacó la importancia de la estacionalidad en la inflación mexicana, basada en los hallazgos de Capistrán, Constandse, y Ramos-Francia[5], quienes identificaron que la inflación es explicada en gran medida por su componente estacional.

En su investigación, Pedroza Robles propuso una arquitectura basada en el modelo de Nakamura [18] y entrenó la red neuronal por 100 épocas. Para evaluar su desempeño, utilizó métricas como el error cuadrático medio (0.000522) y la pérdida empírica (0.027049), demostrando que el modelo logró generalizar patrones dentro de la serie de tiempo.

Este estudio evidencia que las redes neuronales son una herramienta viable para la predicción de la inflación en México, resaltando la importancia de considerar la estacionalidad dentro de los modelos predictivos.

3 Fundamentos Teóricos

3.1 Inflación

Antes de comenzar con detalles técnicos sobre las herramientas y los modelos que serán utilizados para el pronóstico de la inflación, es necesario entender que es y que la causa.

¿Qué es la inflación? La inflación puede entenderse como la depreciación de nuestro dinero con el tiempo. Es decir, lo que antes nos costaba un producto, ahora este cuesta más, o por el contrario, la cantidad que solíamos obtener de un bien o servicio con 10 pesos, ahora es menor. El ejemplo más claro es la gasolina; en el año 2005 1 litro costaba 12 pesos, mientras que en 2025, el litro esta entre 21 y 25 pesos, dependiendo la zona en la que nos encontremos.

¿Qué causa la inflación? De acuerdo con la información encontrada en el portal de bbva [3] hay 4 razones. La primera se relaciona con la interacción entre la oferta y la demanda de un bien cuyo precio se determina libremente en el mercado. Supóngase el caso de un producto de consumo no regulado, como el aguacate. Si durante un mes la demanda alcanza las 200 unidades, mientras que la producción disponible es únicamente de 100 unidades, se genera un exceso de demanda. Bajo estas condiciones, el precio del producto tenderá a incrementarse. En contraste, si la producción mensual superara ampliamente la demanda; por ejemplo, con una oferta de 300 unidades, la presión sobre los precios sería menor. Cuando este tipo de incrementos de precios se presenta de manera persistente en el tiempo, se manifiesta como un proceso inflacionario.

La segunda razón viene al haber un incremento en los costos de producción. El ejemplo más claro en 2025 es el aumento en los aranceles de 25 %, lo que hace que se encarezcan las materias primas, y por lo tanto, el precio final del producto. Ya que de no ser así, la empresa estaría perdiendo utilidades.

La tercera razón por la que esto ocurre se debe al crecimiento de la cantidad

de dinero legal que se produce, esto significa que hay más dinero en circulación para gastar en bienes y servicios. Lo que nos lleva a un incremento en la demanda, y como vimos en el primer punto, la demanda al ser mayor a la oferta, causa que los bienes y servicios se encarezcan. (No necesariamente todos, como se verá a continuación).

El último punto tiene que ver con políticas internas del gobierno. A este efecto se le conoce como inflación autoconstruida.

¿Qué tipos existen? Existen tres tipos, la modera consiste en un aumento gradual de los precios, y en general, esta no supera el 10 % anual. Por otra parte está la galopante, donde el aumento de los precios es muy elevada, hablando de un 15 % a 130 % anual, lo que hace que se reduzca el poder adquisitivo de los consumidores, lo que afecta directamente a la economía de un país, hecho que es alarmante. Finalmente esta la hiperinflación, donde los precios ascienden de manera brutal, lo que llevaría a cualquier país a una crisis económica. Afortunadamente no es el caso de México, de hecho, se tiene una inflación moderada y como se verá en el siguiente capítulo, la inflación en México exhibe un comportamiento estacionario.

3.2 Series Temporales (conceptos base)

3.2.1 Serie temporal

Definición: Una **serie temporal** es una secuencia de observaciones ordenadas en el tiempo. Formalmente, puede representarse como un proceso estocástico $\{x_t : t \in T\}$, donde x_t es una variable aleatoria que describe el valor del sistema en el tiempo t y T es un conjunto ordenado de instantes temporales [26].

En el contexto de esta tesis, la serie de interés es la inflación mensual en México, cuyo comportamiento histórico se muestra en la Figura 2.



Figura 2: Comportamiento histórico de la inflación en México.

3.2.2 Esperanza

Definición: El valor esperado de un proceso $\{X_t\}$ se define como

$$\mu_t = \mathbb{E}[X_t] = \int_{-\infty}^{\infty} x f_{X_t}(x) dx,$$

dado que la integral exista [26].

En series temporales, μ_t puede ser constante o depender del tiempo. Cuando la esperanza varía con t , esta función se interpreta como la **tendencia** o **nivel** del proceso.

3.2.3 Varianza

La **varianza** de un proceso $\{X_t\}$ describe la dispersión de sus valores alrededor de la media. Para un tiempo dado t , la varianza se define como:

$$\text{Var}(X_t) = \mathbb{E}[(X_t - \mu_t)^2].$$

Si la varianza es constante, el proceso es **homocedástico**; si varía con el tiempo, se denomina **heterocedástico**. En el contexto de la inflación, la varianza refleja la **volatilidad** del proceso: periodos con choques externos (crisis financieras, perturbaciones energéticas, políticas monetarias) suelen generar incrementos temporales en la varianza aun cuando la tendencia general se mantenga.

3.2.4 Autocovarianza

Definición: La autocovarianza de un proceso $\{X_t\}$ está dada por

$$\gamma(s, t) = \text{Cov}(X_s, X_t) = \mathbb{E}[(X_s - \mu_s)(X_t - \mu_t)],$$

para todo par de instantes s y t [26].

Esta función cuantifica la dependencia lineal entre dos observaciones en distintos tiempos. Valores altos indican que la serie mantiene memoria a través del tiempo, mientras que valores cercanos a cero reflejan débil dependencia lineal.

Cabe notar que $\gamma(s, t) = 0$ implica ausencia de relación lineal, pero no excluye la posibilidad de dependencias no lineales.

3.2.5 Autocorrelación

La **función de autocorrelación** (ACF) se define como

$$\rho(s, t) = \frac{\gamma(s, t)}{\sqrt{\gamma(s, s) \gamma(t, t)}}.$$

La autocorrelación es simplemente la autocovarianza normalizada, acotada entre -1 y 1 , y permite comparar dependencias temporales entre series con distintas escalas [26].

La Figura 3 muestra la ACF de la inflación mexicana, donde se observa una fuerte persistencia en los primeros rezagos, justificando el uso de modelos capaces de capturar dependencias temporales, tales como RNN, LSTM y arquitecturas basadas en atención.

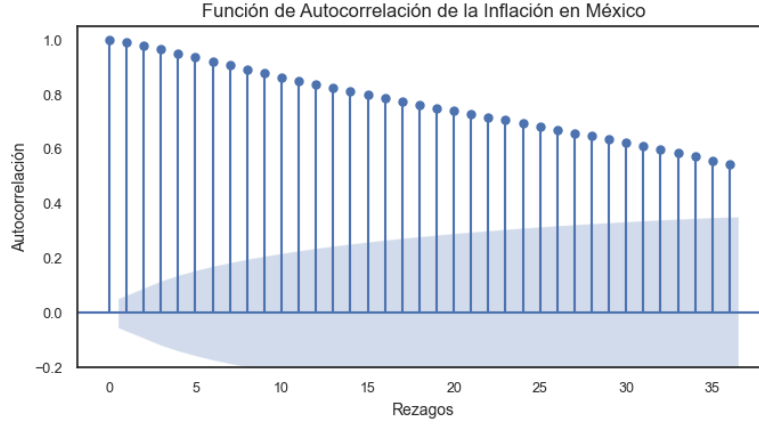


Figura 3: Función de autocorrelación (ACF) de la inflación en México.

3.2.6 Estacionariedad

Una serie temporal es **débilmente estacionaria** si satisface tres condiciones fundamentales [26]:

1. Su media μ_t es constante en el tiempo.
2. Su varianza $\text{Var}(X_t)$ es constante y finita.
3. Su función de autocovarianza depende únicamente del desfase $h = |t - s|$, es decir,

$$\gamma(s, t) = \gamma(h).$$

En la práctica, muchas series económicas —incluida la inflación— violan estas propiedades debido a tendencias, ciclos prolongados o cambios estructurales. Por ello, antes de modelarlas es común aplicar transformaciones que eliminen estos componentes no estacionarios y revelen la estructura dinámica subyacente.

Una forma clásica de modelar la no estacionariedad es:

$$X_t = \mu_t + Y_t,$$

donde μ_t representa la tendencia de largo plazo y Y_t es un proceso estacionario. La presencia de μ_t oculta patrones de corto plazo, por lo que es deseable estimar y remover la tendencia como parte del análisis exploratorio.

Diferenciación Una alternativa práctica consiste en aplicar la diferencia temporal:

$$X_t - X_{t-1} \approx (Y_t - Y_{t-1}) + \delta + w_t,$$

lo que suele producir una señal más cercana a la estacionariedad sin necesidad de especificar paramétricamente la forma de μ_t [9].

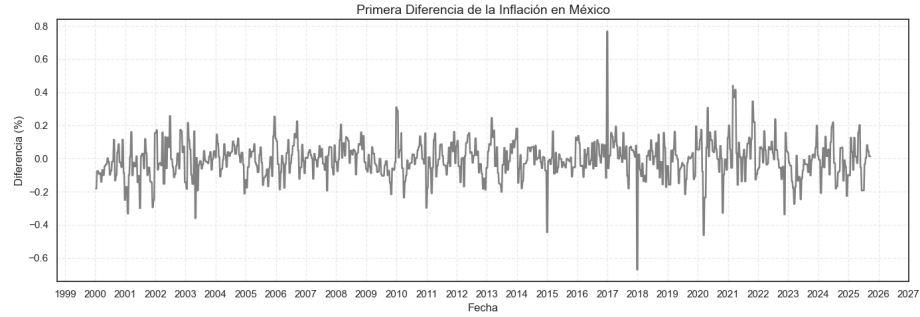


Figura 4: Primera diferencia de la inflación mensual en México, utilizada para inducir estacionariedad.

3.2.7 Descomposición mediante el filtro de Hodrick–Prescott

El filtro de Hodrick-Prescott (HP), introducido por [7], permite separar una serie temporal en dos componentes:

$$X_t = \tau_t + c_t,$$

donde τ_t es una tendencia suavizada y c_t representa el componente cíclico o residual, más cercano a un proceso estacionario. Esta descomposición es especialmente útil en el caso de la inflación, donde las variaciones de largo plazo pueden ocultar la estructura dinámica relevante.

La tendencia se obtiene como la solución del problema de optimización

$$\min_{\{\tau_t\}_{t=1}^T} \left\{ \sum_{t=1}^T (X_t - \tau_t)^2 + \lambda \sum_{t=2}^{T-1} [(\tau_{t+1} - \tau_t) - (\tau_t - \tau_{t-1})]^2 \right\}.$$

El primer término evita que la tendencia se aleje excesivamente de la serie observada, mientras que el segundo penaliza cambios abruptos en su pendiente. El parámetro λ controla el balance entre ambas partes. Siguiendo la regla de ajuste por frecuencia propuesta por [21], se parte de $\lambda = 1600$ para datos trimestrales y se utiliza

$$\lambda_f = 1600 \left(\frac{f}{4} \right)^4,$$

donde f es el número de observaciones por año. Para la frecuencia semanal empleada en esta investigación, $f = 52$ y, por tanto, $\lambda = 45\,697\,600$.

El filtro HP es bilateral: la estimación de la tendencia cercana al final de una muestra es sensible a observaciones posteriores. Para evitar fuga de información, en cada ventana de validación el filtro se ajusta nuevamente y recibe únicamente los datos anteriores o iguales a la fecha de corte. En consecuencia, ninguna observación del horizonte evaluado participa en la descomposición utilizada para entrenar los modelos. Esta política no elimina el conocido sesgo de extremo, pero impide que el pronóstico utilice información futura de forma directa o indirecta.

Una vez obtenidos los componentes, cada arquitectura entrena dos instancias independientes:

$$\hat{\tau}_{t+1:t+h} = f_{\tau}(\tau_{1:t}), \quad \hat{c}_{t+1:t+h} = f_c(c_{1:t}),$$

y el pronóstico final se reconstruye mediante

$$\hat{X}_{t+k} = \hat{\tau}_{t+k} + \hat{c}_{t+k}, \quad k = 1, \dots, h.$$

Las transformaciones descritas en la siguiente subsección se aplican exclusivamente a τ_t . El ciclo se conserva en su escala estructural y únicamente se normaliza con parámetros calculados en el conjunto de entrenamiento.

La Figura 5 muestra la inflación observada, la tendencia HP y el componente cíclico resultante.

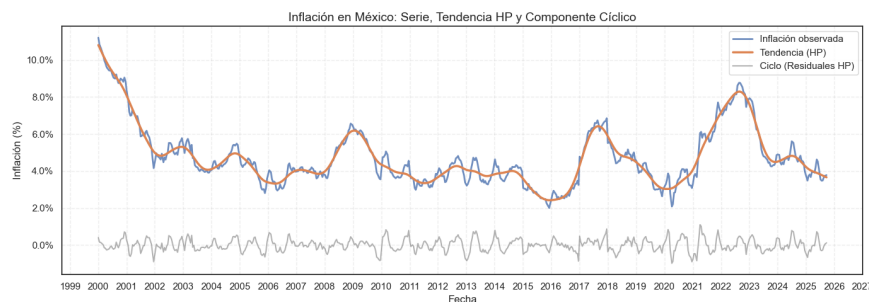


Figura 5: Inflación mensual en México: serie original, tendencia estimada mediante el filtro HP y componente cíclico.

3.2.8 Transformaciones aplicadas

Para inducir estacionariedad y mejorar el desempeño de los modelos predictivos, el framework implementado en este trabajo permite aplicar diversas transformaciones a la serie original. Estas transformaciones estabilizan la varianza, eliminan tendencias o resaltan cambios relativos. Las opciones disponibles incluyen:

1. Diferencia simple (diff)

$$\nabla x_t = x_t - x_{t-1}.$$

Elimina tendencias lineales y resalta variaciones de corto plazo [9].

2. Diferencia del logaritmo amortiguado (diff_logp1)

$$\nabla \log(1 + x_t) = \log(1 + x_t) - \log(1 + x_{t-1}).$$

Reduce el impacto de valores extremos y convierte cambios absolutos en relativos.

3. Cambio porcentual (pct)

$$\frac{x_t - x_{t-1}}{x_{t-1}}.$$

Captura variaciones proporcionales e induce homocedasticidad en diversas series económicas.

4. Transformación logarítmica amortiguada (logp1)

$$\log(1 + x_t).$$

Suaviza la serie y estabiliza la varianza.

5. Segunda diferencia (diff2)

$$\nabla^2 x_t = (x_t - x_{t-1}) - (x_{t-1} - x_{t-2}).$$

Elimina tendencias de orden superior o estructuras suavemente crecientes.

Estas transformaciones fueron implementadas de manera parametrizable dentro del sistema, lo que permite seleccionar la opción más adecuada para cada serie. No obstante, es importante evitar una “sobre-estacionarización”, ya que transformar en exceso la serie puede eliminar información relevante para el proceso generador [9].

3.3 Embedding

Un **embedding** es una representación vectorial en un espacio continuo de dimensión reducida, utilizada para transformar elementos discretos, como categorías, identificadores o índices temporales en vectores de números reales. Su propósito es capturar relaciones latentes entre los elementos de manera que aquellos más similares queden cercanos entre sí dentro del espacio geométrico de representación [16].

Formalmente, un embedding es una función

$$f : \mathcal{C} \rightarrow \mathbb{R}^d,$$

donde $\mathcal{C}$ es un conjunto discreto (por ejemplo, meses, días de la semana, palabras, imágenes o identificadores de series), y d es la dimensión del espacio vectorial continuo.

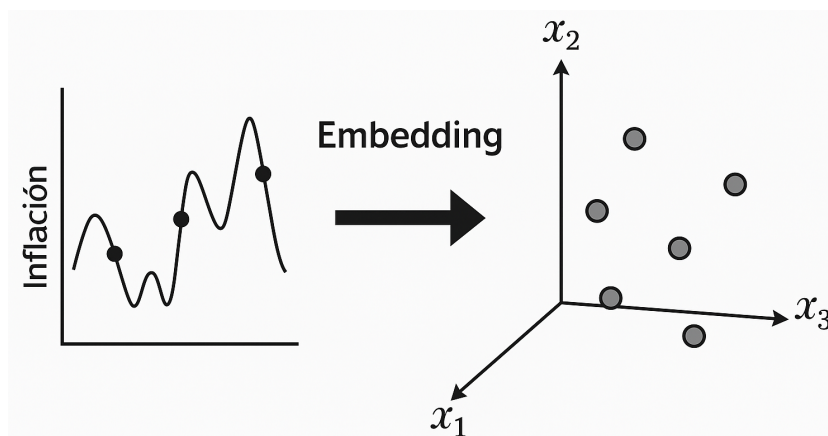


Figura 6: Representación conceptual del embedding: cada observación de la serie de inflación se proyecta a un espacio vectorial tridimensional.

En el contexto de series temporales, los embeddings permiten representar atributos temporales (como el índice de tiempo, el mes del año o identificadores

de series) en un espacio donde los modelos neuronales pueden operar mediante álgebra lineal. Esta representación densa es fundamental en arquitecturas modernas como LSTM, Transformer y D³VAE, las cuales dependen de embeddings para capturar dependencias temporales y relaciones entre series individuales.

3.4 Fundamentos probabilísticos

Muchos de los modelos empleados en este trabajo, en particular DeepAR y D³VAE, se formulan explícitamente en términos probabilísticos. Estos métodos estiman distribuciones de probabilidad condicionales, optimizan funciones de log-verosimilitud y utilizan divergencias entre distribuciones para regularizar espacios latentes o medir discrepancias. Con este fin, se presentan a continuación algunos conceptos fundamentales necesarios para comprender estos modelos [4, 17].

3.4.1 Operaciones básicas sobre distribuciones de probabilidad

Sea $p(x)$ una función de densidad de probabilidad (PDF) definida sobre una variable aleatoria continua x . Algunas operaciones fundamentales en modelos probabilísticos son:

- **Logaritmo de la densidad:**

$$\log p(x).$$

El uso del logaritmo facilita el cálculo numérico y es esencial en el aprendizaje por máxima verosimilitud, donde se optimiza la log-verosimilitud en lugar de la verosimilitud directa [4].

- **Gradiente del logaritmo:**

$$\nabla_x \log p(x) = \frac{\nabla_x p(x)}{p(x)}.$$

Esta cantidad, conocida como *score function*, describe la dirección de máximo incremento relativo de la densidad. Es útil en métodos de optimización y en técnicas de estimación basadas en gradientes [17].

3.4.2 Logaritmo de una distribución normal

Si $x \sim \mathcal{N}(\mu, \sigma^2)$, su densidad es:

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right).$$

El logaritmo de la densidad se obtiene aplicando logaritmo natural:

$$\log p(x) = -\frac{(x-\mu)^2}{2\sigma^2} - \frac{1}{2} \log(2\pi\sigma^2),$$

lo que corresponde a una parábola invertida en x , acompañada de un término constante.

El gradiente de la log-densidad es:

$$\nabla_x \log p(x) = -\frac{x - \mu}{\sigma^2}.$$

Esta expresión indica que la fuerza con la cual la densidad “empuja” a x hacia la media es proporcional a la distancia $x - \mu$, escalada por el inverso de la varianza. Este comportamiento es fundamental en modelos latentes gaussianos y en la inferencia variacional [11].

3.4.3 Divergencia de Kullback–Leibler

La divergencia de Kullback–Leibler ($\mathcal{D}_{KL}$) mide la discrepancia entre dos distribuciones de probabilidad $p(x)$ y $q(x)$:

$$\mathcal{D}_{KL}(p||q) = \int p(x) \log \left(\frac{p(x)}{q(x)} \right) dx.$$

Sus propiedades principales son [14]:

- **No negatividad:** $\mathcal{D}_{KL}(p||q) \geq 0$.
- **Identidad:** $\mathcal{D}_{KL}(p||q) = 0$ si y sólo si $p = q$ casi siempre.
- **Asimetría:** $\mathcal{D}_{KL}(p||q) \neq \mathcal{D}_{KL}(q||p)$.

En modelos generativos como las VAEs, la divergencia KL actúa como un término de regularización que controla cuán cerca se mantiene la distribución aproximada del posterior respecto a un *prior* elegido, jugando un papel fundamental en la calidad del espacio latente [11].

Estas herramientas probabilísticas serán utilizadas posteriormente al describir modelos como DeepAR y D³VAE, los cuales formulan el problema de pronóstico como la estimación de una distribución condicional y emplean funciones de pérdida basadas en log-verosimilitud o divergencias entre distribuciones.

3.5 Transformación Discreta de Fourier (DFT)

La Transformación Discreta de Fourier (DFT) es una herramienta matemática que permite descomponer una serie temporal como una combinación de componentes senoidales de distintas frecuencias. Su propósito es representar la señal en el dominio de la frecuencia, lo cual facilita la identificación de patrones cíclicos, estacionalidades u oscilaciones dominantes [22].

Dada una secuencia de observaciones

$$x_0, x_1, \dots, x_{N-1},$$

la DFT se define como:

$$X_k = \sum_{n=0}^{N-1} x_n e^{-2\pi i \frac{kn}{N}}, \quad k = 0, 1, \dots, N-1. \quad (1)$$

Aquí, X_k representa la amplitud y fase asociadas a la frecuencia k . Esta representación permite reconstruir la señal original mediante la transformada inversa:

$$x_n = \frac{1}{N} \sum_{k=0}^{N-1} X_k e^{2\pi i \frac{kn}{N}}. \quad (2)$$

En el presente trabajo, la DFT se aplica a la serie inflacionaria con el propósito de identificar las frecuencias de mayor energía y utilizarlas para construir variables exógenas senoidales. A partir de las k frecuencias dominantes se generan funciones seno y coseno normalizadas que capturan estacionalidades explícitas. Estas variables son integradas posteriormente en modelos como XGBoost, LSTM, RNN, DeepAR y Transformer.

Este procedimiento aporta información periódica sin requerir que los modelos aprendan todas las estacionalidades de manera implícita, mejorando su capacidad para capturar patrones recurrentes en escalas largas y medianas.

3.6 Fundamentos estadísticos del aprendizaje

El presente capítulo tiene como propósito establecer los principios matemáticos y estadísticos que sustentan el aprendizaje automático aplicado en esta investigación. Estos fundamentos permiten entender los mecanismos detrás del proceso de modelado, la naturaleza de los errores de predicción y las métricas utilizadas para evaluar el desempeño de los algoritmos.

3.6.1 Formulación general del problema de aprendizaje

Sea un conjunto de observaciones $\{(x_i, y_i)\}_{i=1}^n$, donde cada $x_i \in \mathbb{R}^p$ representa un vector de predictores o variables explicativas y $y_i \in \mathbb{R}$ la variable de respuesta. Se asume la existencia de una función desconocida f que relaciona ambas cantidades:

$$Y = f(X) + \epsilon,$$

donde ϵ representa un término de error aleatorio con media cero, $\mathbb{E}[\epsilon] = 0$, y varianza constante $\text{Var}(\epsilon) = \sigma^2$. El objetivo del aprendizaje estadístico consiste en estimar una función $\hat{f}$ que aproxime de la mejor manera posible a f , minimizando la discrepancia entre las predicciones $\hat{y}_i = \hat{f}(x_i)$ y los valores reales y_i .

Este proceso puede entenderse como una búsqueda en el espacio de funciones posibles $\mathcal{F}$, de modo que:

$$\hat{f} = \arg \min_{f \in \mathcal{F}} \mathbb{E}[(Y - f(X))^2].$$

En la práctica, la esperanza no se conoce, por lo que se utiliza una versión empírica basada en el conjunto de entrenamiento disponible.

3.6.2 Error reducible e irreducible

El error esperado de predicción en un punto x_0 se descompone de la siguiente forma:

$$\mathbb{E}[(Y - \hat{f}(X))^2] = \underbrace{\mathbb{E}[(f(X) - \hat{f}(X))^2]}_{\text{Error reducible}} + \underbrace{\text{Var}(\epsilon)}_{\text{Error irreducible}}.$$

El *error reducible* depende de la capacidad del modelo para aproximar correctamente la función f . Mediante una mejor arquitectura, mayor cantidad de datos o un ajuste adecuado de hiperparámetros, este término puede disminuirse. Por otro lado, el *error irreducible* corresponde a la variabilidad inherente del proceso generador de los datos; es decir, al ruido que no puede ser explicado por ninguna función de X . Este componente establece un límite inferior en la exactitud de cualquier modelo predictivo. [10]

3.6.3 Descomposición sesgo-varianza

Para un punto nuevo x_0 , el error cuadrático medio esperado puede descomponerse como:

$$\mathbb{E}[(y_0 - \hat{f}(x_0))^2] = \text{Var}(\hat{f}(x_0)) + [\text{Bias}(\hat{f}(x_0))]^2 + \text{Var}(\epsilon).$$

El segundo término mide el *sesgo* (bias), es decir, la diferencia sistemática entre el valor esperado de la predicción y el valor real. El primero representa la *varianza*, que indica qué tanto cambia la predicción ante distintos conjuntos de entrenamiento.

Modelos muy flexibles (como las redes neuronales profundas) suelen tener bajo sesgo pero alta varianza, mientras que modelos rígidos (como las regresiones lineales) presentan el comportamiento opuesto. El objetivo del aprendizaje estadístico es encontrar un punto de equilibrio donde tanto el sesgo como la varianza sean bajos, logrando así una buena generalización en datos no observados. [10]

3.6.4 Métricas de evaluación

La evaluación de un modelo requiere cuantificar qué tan cercanas son sus predicciones a los valores reales. Entre las métricas más comunes utilizadas en esta tesis se encuentran:

$$\begin{aligned} \mathbf{MAE} &= \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|, & \mathbf{MSE} &= \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2, \\ \mathbf{RMSE} &= \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}, & \mathbf{MAPE} &= \frac{100}{n} \sum_{i=1}^n \frac{|y_i - \hat{y}_i|}{|y_i|}. \end{aligned}$$

En el presente estudio se adopta el **Error Absoluto Medio (MAE)** como métrica principal de comparación, debido a su robustez frente a valores atípicos y su interpretación directa en unidades de inflación. Las demás métricas se incluyen con fines complementarios y de contraste en el análisis de resultados.

3.6.5 Error de predicción y evaluación fuera de muestra

Durante el entrenamiento, el modelo se ajusta para minimizar el error sobre el conjunto de entrenamiento. Sin embargo, el interés real radica en medir su desempeño en observaciones no vistas previamente. Sea (x_0, y_0) una nueva observación, la capacidad predictiva se evalúa a través del error:

$$e_0 = y_0 - \hat{f}(x_0),$$

y su valor absoluto medio a lo largo de todo el conjunto de prueba. Un modelo con bajo error en entrenamiento pero alto error en prueba se considera sobreajustado, mientras que un modelo con errores altos en ambos conjuntos probablemente sea insuficientemente flexible.

3.6.6 Linealidad y mínimos cuadrados

En el contexto de predicción de series temporales, un modelo puede perder utilidad si sus predicciones son excesivamente lineales. Para evaluar este fenómeno, se ajusta la recta que mejor aproxima las predicciones mediante el método de mínimos cuadrados lineales, resolviendo:

$$\min_{m,b} \sum_t (\hat{y}_t - (mt + b))^2,$$

donde m es la pendiente y b la ordenada al origen. El ajuste se realiza a través del sistema matricial:

$$A = \begin{bmatrix} 0 & 1 \\ 1 & 1 \\ 2 & 1 \\ \vdots & \vdots \\ n-1 & 1 \end{bmatrix}, \quad A^\top A \begin{bmatrix} m \\ b \end{bmatrix} = A^\top \hat{y}.$$

El error cuadrático medio entre la línea ajustada y las predicciones permite cuantificar el grado de linealidad:

$$\text{RMSE}_{\text{lin}} = \sqrt{\frac{1}{n} \sum_t (\hat{y}_t - (mt + b))^2}.$$

Si este valor es muy bajo, se considera que el modelo está generando pronósticos lineales y, por tanto, poco informativos respecto a una regresión o promedio histórico. Este criterio se implementa en los experimentos como un filtro adicional para evitar configuraciones triviales.

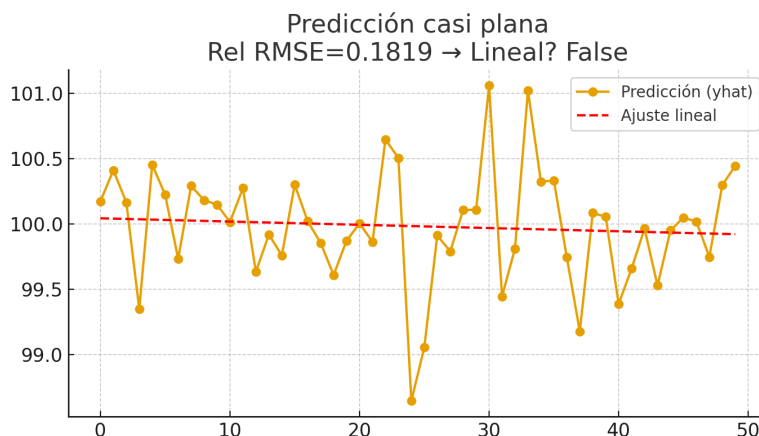


Figura 7: Detección de Pronósticos Lineales

4 Espectro de Modelos de Pronóstico de Inflación: De Enfoques Simples a Arquitecturas Avanzadas de Aprendizaje Profundo

4.1 Naive

Este método es el más simple de todos, consiste en tomar el último valor conocido en una serie temporal, como la predicción para el valor futuro y_{t+1}

$$\hat{y}_{t+1} = y_t. \quad (3)$$

4.2 Promedio

Otro método simple es usar el promedio de todos los valores conocidos en la serie temporal hasta el momento t , y tomar este valor como la predicción futura. Es decir,

$$\hat{y}_{t+1} = \frac{1}{t} \sum_{i=1}^t y_i. \quad (4)$$

4.3 Suavizado Exponencial Simple

Este modelo propone asignarle un peso a todos los valores de la serie temporal, de tal manera que los valores más cercanos al tiempo t tendrán más peso que aquellos valores lejanos a t .

$$\hat{y}_{t+1} = \alpha y_t + (1 - \alpha) \hat{y}_t. \quad (5)$$

Si se desarrolla la parte derecha de la ecuación anterior, y se reemplaza recursiva-mente $\hat{y}_{t-1}$, se obtiene una expansión infinita ponderada de valores pasados. Por lo tanto al desarrollar se tiene lo siguiente...

$$\hat{y}_{t+1} = \alpha y_t + (1 - \alpha) \hat{y}_t \quad (6)$$

$$= \alpha y_t + (1 - \alpha) [\alpha y_{t-1} + (1 - \alpha) \hat{y}_{t-1}] \quad (7)$$

$$= \alpha y_t + \alpha(1 - \alpha) y_{t-1} + (1 - \alpha)^2 \hat{y}_{t-1} \quad (8)$$

$$= \alpha y_t + \alpha(1 - \alpha) y_{t-1} + (1 - \alpha)^2 [\alpha y_{t-2} + (1 - \alpha) \hat{y}_{t-2}] \quad (9)$$

$$= \alpha y_t + \alpha(1 - \alpha) y_{t-1} + \alpha(1 - \alpha)^2 y_{t-2} + (1 - \alpha)^3 \hat{y}_{t-2} \quad (10)$$

$$\vdots$$

$$= \sum_{i=0}^{\infty} \alpha(1 - \alpha)^i y_{t-i}. \quad (11)$$

De acuerdo con [19], el suavizado exponencial simple no es bueno cuando los datos presentan tendencia. El coeficiente α simplemente no es suficiente.

4.4 Suavizado Exponencial Doble (Holt)

4.4.1 Nivel

El nivel, no es más que el valor esperado. Este se puede interpretar como

4.4.2 Tendencia o Pendiente

Es una operación que hemos visto a lo largo de toda nuestra vida, esta se calcula como

$$m = \frac{\Delta y}{\Delta t}. \quad (12)$$

Donde Δy es la diferencia de los valores observados y , y Δt es la diferencia de los momentos t de acuerdo con los valores observados de y .

Como su nombre lo dice, se aplica suavizado exponencial a dos componentes, al nivel ℓ_t y la tendencia b_t , los cuales se definen a continuación.

$$\ell_t = \alpha y_t + (1 - \alpha)(\ell_{t-1} + b_{t-1}) \quad (13)$$

$$b_t = \beta(\ell_t - \ell_{t-1}) + (1 - \beta)b_{t-1}. \quad (14)$$

Por lo tanto, para valores futuros se tiene...

$$\hat{y}_{t+1} = \ell_t + b_t$$

Note que ℓ_t es lo mismo que la ecuación 5, con la diferencia de que se multiplica la constante $(1 - \alpha)$ por la suma del valor esperado pasado ℓ_{t-1} más la tendencia pasada b_{t-1} .

Vamos a desarrollar ℓ_t y b_t recursivamente para entender cómo todo depende de las observaciones pasadas $y_t, y_{t-1}, \dots$

$$\ell_t = \alpha y_t + (1 - \alpha)(\ell_{t-1} + b_{t-1}) \quad (15)$$

$$= \alpha y_t + (1 - \alpha) [\alpha y_{t-1} + (1 - \alpha)(\ell_{t-2} + b_{t-2}) + b_{t-1}] \quad (16)$$

$$= \alpha y_t + \alpha(1 - \alpha)y_{t-1} + (1 - \alpha)^2(\ell_{t-2} + b_{t-2}) + (1 - \alpha)b_{t-1}. \quad (17)$$

De manera similar, para la tendencia:

$$b_t = \beta(\ell_t - \ell_{t-1}) + (1 - \beta)b_{t-1} \quad (18)$$

$$= \beta(\ell_t - \ell_{t-1}) + (1 - \beta) [\beta(\ell_{t-1} - \ell_{t-2}) + (1 - \beta)b_{t-2}] \quad (19)$$

$$= \beta(\ell_t - \ell_{t-1}) + \beta(1 - \beta)(\ell_{t-1} - \ell_{t-2}) + (1 - \beta)^2 b_{t-2}. \quad (20)$$

Esto significa que tanto el nivel como la tendencia se construyen como combinaciones suavizadas de observaciones pasadas y de sus propios estados anteriores. Si seguimos expandiendo, veremos que ambos se expresan como una especie de media ponderada exponencial de diferencias y valores pasados, aunque más compleja que en el suavizado exponencial simple.

Por lo tanto, el pronóstico $\hat{y}_{t+h}$ puede interpretarse como una extrapolación lineal ajustada dinámicamente:

$$\hat{y}_{t+h} = \ell_t + hb_t$$

donde ℓ_t y b_t ya encapsulan toda la información histórica de y_t .

De acuerdo con [20], para definir los valores iniciales de la tendencia, se tienen las siguientes opciones.

$$b_1 = y_2 - y_1 \quad (21)$$

$$b_1 = \frac{1}{3} [(y_2 - y_1) + (y_3 - y_2) + (y_4 - y_3)] \quad (22)$$

$$b_1 = \frac{y_n - y_1}{n - 1}. \quad (23)$$

Mientras que el valor esperado suele ser $\ell_1 = y_1$, tal como ocurre en el suavizado exponencial simple.

4.5 Suavizado Exponencial Triple - Holt Winters

Si la serie temporal que se está buscando modelar presenta tendencia y estacionalidad, el suavizado exponencial doble no funcionará, mucho menos el simple. De acuerdo con [27], la idea detrás del suavizado exponencial triple es aplicar el suavizado exponencial a tres factores, los componentes estacionales s , el valor esperado ℓ y la tendencia b . El resultado de las siguientes ecuaciones es conocido como el método "Holt Winters", este nombre sale de sus inventores.

4.5.1 Estación

En el contexto de series temporales, se puede definir a una estación como la repetición de un patrón dado un cierto intervalo de pasos. Si una serie presenta estacionalidad, ya sea por la época del año, descuentos u otra variable predictora, el método de Holt Winters se vuelve en una buena opción para modelar este tipo de series.

4.5.2 Longitud de una Estación

La longitud de una estación es la cantidad de pasos requeridos para que una nueva estación comience. En general, para datos semanales, se puede tener una longitud de $52,14 \approx 52$.

4.5.3 Componentes Estacionales

Un componente estacional sale del valor esperado más la tendencia que se repiten al mismo tiempo en la estación. Existe un componente estacional por cada punto en una estación. Por lo que si se tiene una estacionalidad de 12 meses, entonces habrá 12 componentes estacionales. Si los datos son agrupados de forma semanal, entonces se tendrá un total de 52 componentes estacionales. [27]

4.5.4 Adición vs Multiplicación

La tendencia en una serie temporal puede ser explicada de forma aditiva o multiplicativa, es decir, en lugar de substraer y_{t-1} de y_t , se pueden dividir y obtener en su lugar el porcentaje de cambio. Por un lado puedes decir que algo ahora cuesta 30 pesos más, y por otro, que aumento el precio en un 10%. Esto es lo que da paso a que existan dos variantes del método de Holt Winters.

La versión aditiva se define de la siguiente manera y se usa cuando la estacionalidad tiene una variación aproximadamente constante en el tiempo.

$$\ell_t = \alpha(y_t - s_{t-L}) + (1 - \alpha)(\ell_{t-1} + b_{t-1}) \quad (24)$$

$$b_t = \beta(\ell_t - \ell_{t-1}) + (1 - \beta)b_{t-1} \quad (25)$$

$$s_t = \gamma(y_t - \ell_{t-1} - b_{t-1}) + (1 - \gamma)s_{t-L} \quad (26)$$

$$\hat{y}_{t+h} = \ell_t + hb_t + s_{t+h-L(k+1)}. \quad (27)$$

donde k es la parte entera de $\frac{h-1}{m}$, la cual asegura que las estimaciones de los índices estacionales usados para pronosticar provengan de la última estación de la muestra. Véase que la ecuación del nivel ℓ_t es un promedio ponderado entre el valor observado y la estacionalidad ajustada ($y_t - s_{t-L}$) y el pronostico no estacional ($\ell_{t-1} + b_{t-1}$) para el momento t . Por otro lado, la ecuación correspondiente a la tendencia b_t permanece igual que en el método de Holt. Mientras que la ecuación de estacionalidad s_t es el promedio ponderado entre ($y_t - \ell_{t-1} - b_{t-1}$) y el índice estacional correspondiente al periodo pasado s_{t-L} . [8]

Mientras que la versión multiplicativa se usa cuando la estacionalidad tiene una variación proporcional al nivel de la serie. [8].

$$l_t = \alpha \left(\frac{y_t}{s_{t-L}} \right) + (1 - \alpha)(l_{t-1} + b_{t-1}) \quad (28)$$

$$b_t = \beta^*(l_t - l_{t-1}) + (1 - \beta^*)b_{t-1} \quad (29)$$

$$s_t = \gamma \frac{y_t}{l_t} + (1 - \gamma)s_{t-L} \quad (30)$$

$$\hat{y}_{t+h} = (l_t + hb_t)s_{t+h-L(k+1)}. \quad (31)$$

Donde α, β^*, γ son parámetros de suavizado ($0 < \alpha, \beta^*, \gamma < 1$) y m es la longitud del período estacional.

Para poder usar el método de Holt Winters es necesario poseer por lo menos una estacionalidad completa para determinar los valores iniciales de los índices de estacionalidad s_{t-L} . Una estación completa de observaciones consiste de L periodos. Y se desea estimar la tendencia de un periodo a otro. Para lograr esto, es necesario usar dos estaciones completas, es decir, $2L$ periodos/observaciones.

Los valores iniciales para la tendencia se obtienen a partir de la siguiente formula

$$b = \frac{1}{L} \left(\frac{y_{L+1} - y_1}{L} + \frac{y_{L+2} - y_2}{L} + \dots + \frac{y_{L+L} - y_L}{L} \right). \quad (32)$$

Mientras que los valores iniciales de los índices estacionales se calculan a partir de la media de cada estación y de la media global de las primeras N estaciones completas. Para el modelo aditivo, los índices estacionales se definen como:

$$s_i = \frac{1}{N} \sum_{j=1}^N (y_{L(j-1)+i} - \bar{y}_j), \quad i = 1, 2, \dots, L. \quad (33)$$

donde $\bar{y}_j$ representa la media de la j -ésima estación, y L es la longitud de la estación (por ejemplo, 12 para datos mensuales o 52 para datos semanales). En el caso del modelo multiplicativo, los índices estacionales se obtienen dividiendo cada observación entre la media de su estación:

$$s_i = \frac{1}{N} \sum_{j=1}^N \frac{y_{L(j-1)+i}}{\bar{y}_j}, \quad i = 1, 2, \dots, L. \quad (34)$$

De esta manera, los valores iniciales de tendencia, nivel y estacionalidad permiten al método de Holt-Winters comenzar el proceso iterativo de actualización y pronóstico. En cada paso, los parámetros se ajustan mediante las ecuaciones de suavizado exponencial, dando más peso a las observaciones recientes, lo que confiere al modelo una respuesta adaptativa ante cambios estructurales en la serie temporal.

Árboles de Decisión

Los *Árboles de Decisión* son aplicados a problemas de regresión y clasificación. Y hay una gran variedad de algoritmos basados en árboles. Éstos implican la segmentación del espacio predictor en un número de regiones simples (rectangulares). Para hacer una predicción en una observación x_0 , toman el promedio o la moda de la variable de respuesta de la región para la cual la observación x_0 pertenece. Los árboles de decisión son simples y fáciles de interpretar. Algoritmos como *bagging*, *random forests*, *boosting* y *árboles de regresión aditivos bayesianos* producen múltiples árboles, los cuales se combinan para crear una predicción final. Combinar una gran cantidad de árboles mejoran la capacidad predictora de estos algoritmos, pero se pierde en interpretabilidad.

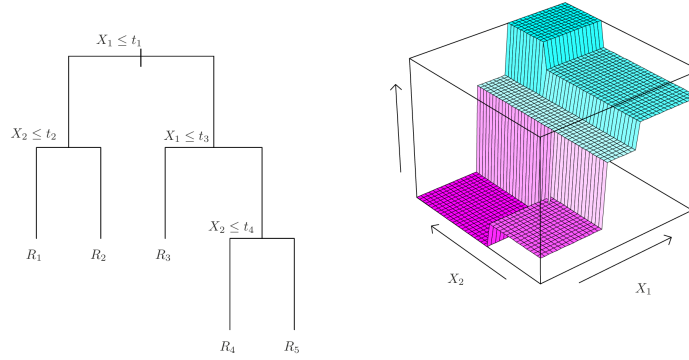


Figura 8: Ejemplo de un árbol de decisión proveniente de [10].

4.6 XGBoost

XGBoost es un ensamble de K árboles, donde cada uno contribuye de manera aditiva a la predicción final con una estructura como la que se muestra en la figura 8, y la predicción se obtiene como:

$$\hat{y}_i = \phi(x_i) = \sum_{k=1}^k f_k(x_i), \quad f_k \in \mathcal{F}. \quad (35)$$

El espacio de funciones $\mathcal{F}$ está definido como:

$$\mathcal{F} = f(x) = w_{q(x)}, \quad (36)$$

Donde $q : \mathbb{R}^m \rightarrow T$ es una función que asigna una observación $\mathbf{x}$ a una de las T hojas del árbol, mientras que $w \in \mathbb{R}^T$ es el vector de pesos asignados a cada hoja. Por lo tanto, cada función f_k representa un árbol con una estructura q y valores de predicción w en sus hojas. [6]

4.6.1 Aprendizaje de Árboles

Para aprender el conjunto de funciones usadas en el modelo, se minimiza la función objetivo regularizado...

$$\mathcal{L}(\phi) = \sum_i \ell(y_i, \hat{y}_i) + \sum_k \Omega(f_k), \quad (37)$$

donde ℓ es una función de pérdida convexa diferenciable, la cual mide la diferencia entre las predicciones $\hat{y}_i$ y la variable de respuesta y_i , mientras que el termino Ω penaliza la complejidad del modelo como se muestra:

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda |w|^2. \quad (38)$$

Esto tiene como consecuencia el resultado de árboles más simples, lo que ayuda a combatir el sobreajuste al suavizar los valores w a través del parámetro de regularización λ .

4.6.2 Gradient Tree Boosting

El modelo se entrena de forma aditiva, por lo que en cada iteración t , se añade una nueva función f_t , de tal manera que se minimice la función de pérdida.

$$\mathcal{L}^{(t)} = \sum_{i=1}^n \ell(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t) \quad (39)$$

Los autores [6] emplearon una aproximación de segundo orden para optimizar de forma rápida la función objetivo.

$$\mathcal{L}^{(t)} \simeq \sum_{i=1}^n \left[\ell(y_i, \hat{y}_i^{(t-1)}) + g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \Omega(f_t), \quad (40)$$

donde $g_i = \partial_{\hat{y}^{(t-1)}} \ell(y_i, \hat{y}_i^{(t-1)})$ y $h_i = \partial_{\hat{y}^{(t-1)}}^2 \ell(y_i, \hat{y}_i^{(t-1)})$ son el gradiente de primer y segundo orden (hessiano) de la función de pérdida ℓ . Véase que el primer termino indica la dirección de corrección, mientras que el segundo, la magnitud de la pérdida. Al quitar los términos constantes $\ell(y_i, \hat{y}_i^{(t-1)})$ se obtiene la ecuación simplificada...

$$\tilde{\mathcal{L}}^{(t)} = \sum_i^n \left[g_i f_t(\mathbf{x}_i) + \frac{1}{2} h_i f_t^2(\mathbf{x}_i) \right] + \Omega(f_t). \quad (41)$$

Luego, al establecer las observaciones correspondientes a cada hoja, los autores [6] definen a $I_j = \{i | q(x_i) = j\}$, y al expandir el termino de regularización, se obtiene...

$$\tilde{\mathcal{L}}^{(t)} = \sum_{j=1}^T \left[\left(\sum_{i \in I_j} g_i \right) w_j + \frac{1}{2} \left(\sum_{i \in I_j} h_i + \lambda \right) w_j^2 \right] + \gamma T. \quad (42)$$

4.6.3 Evaluación de la estructura de un árbol y la división del espacio

Consideré ahora la estructura de un árbol fijo $q(x)$, ahora, se puede calcular el valor óptimo de sus respectivos pesos w_j para cada hoja j de la siguiente manera

$$w_j = - \frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda}. \quad (43)$$

Luego, al reemplazarlos en la función objetivo regularizada, se puede calcular el valor optimo como

$$\tilde{\mathcal{L}}^{(t)}(q) = \sum_{j=1}^T \left[\left(\sum_{i \in I_j} g_i \right) \left(- \frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda} \right) \right] \quad (44)$$

$$+ \frac{1}{2} \left(\sum_{i \in I_j} h_i + \lambda \right) \left(- \frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda} \right)^2 \right] + \gamma T \quad (45)$$

$$= \sum_{j=1}^T \left[\left(- \frac{(\sum_{i \in I_j} g_i)^2}{\sum_{i \in I_j} h_i + \lambda} \right) + \frac{1}{2} \left(\frac{(\sum_{i \in I_j} g_i)^2}{\sum_{i \in I_j} h_i + \lambda} \right) \right] + \gamma T \quad (46)$$

$$= \sum_{j=1}^T \left[\left(-1 + \frac{1}{2} \right) \left(\frac{(\sum_{i \in I_j} g_i)^2}{\sum_{i \in I_j} h_i + \lambda} \right) \right] + \gamma T \quad (47)$$

$$= -\frac{1}{2} \sum_{j=1}^T \frac{(\sum_{i \in I_j} g_i)^2}{\sum_{i \in I_j} h_i + \lambda} + \gamma T. \quad (48)$$

Esta expresión permite saber la calidad de la estructura de un árbol q . En la practica, se usa para medir la impureza de los arboles de decisión, pero basada en derivadas de la función de pérdida, por lo que es aplicable a una gran variedad de funciones objetivo.

Como es computacionalmente costoso evaluar todas las posibles variantes de un árbol, los autores [6] emplean un algoritmo avaricioso, que construye los árboles de manera iterativa. Por lo tanto, en cada paso, se evalúan las posibles divisiones de un nodo actual. Suponiendo que I_R e I_L son los conjuntos de instancias que irían a los nodos izquierdo y derecho tras una división. Si $I = I_L \cup I_R$ representa las instancias actuales del nodo antes de dividirlo, entonces la reducción en la función de pérdida se puede calcular como

$$\mathcal{L}_{\text{split}} = \frac{1}{2} \left[\frac{(\sum_{i \in I_L} g_i)^2}{\sum_{i \in I_L} h_i + \lambda} + \frac{(\sum_{i \in I_R} g_i)^2}{\sum_{i \in I_R} h_i + \lambda} - \frac{(\sum_{i \in I} g_i)^2}{\sum_{i \in I} h_i + \lambda} \right] - \gamma. \quad (49)$$

Esta fórmula se utiliza en la práctica como criterio para evaluar los candidatos de división. Al final, se selecciona la división que maximiza $\mathcal{L}_{\text{split}}$, es decir, la que mayor reducción de pérdida logre.

Ejemplo numérico

Supóngase una división con las siguientes observaciones y valores:

Instancia	g_i	h_i
1	-2,0	1,0
2	-1,0	1,0
3	1,0	1,0

Sea $I_L = \{1, 2\}$, $I_R = \{3\}$, y $\lambda = 1$, $\gamma = 0$. Se calculan:

$$\begin{aligned} \sum_{i \in I_L} g_i &= -3,0, & \sum_{i \in I_L} h_i &= 2,0 \\ \sum_{i \in I_R} g_i &= 1,0, & \sum_{i \in I_R} h_i &= 1,0 \\ \sum_{i \in I} g_i &= -2,0, & \sum_{i \in I} h_i &= 3,0 \end{aligned}$$

Sustituyendo en la ecuación:

$$\mathcal{L}_{\text{split}} = \frac{1}{2} \left[\frac{9}{3} + \frac{1}{2} - \frac{4}{4} \right] = \frac{1}{2} (3 + 0,5 - 1) = \frac{1}{2} (2,5) = 1,25.$$

Como el valor es positivo, la división mejora la función objetivo, y se acepta.

4.6.4 Interpretación de g_i , h_i y w_j

- g_i : representa cuánto cambia la pérdida si se ajusta ligeramente la predicción. Un valor grande indica una predicción muy alejada del valor real.
- h_i : corresponde a la curvatura local de la función de pérdida, permitiendo un ajuste más preciso.
- w_j : es el score asignado a la hoja j , calculado como un promedio ponderado de los gradientes ajustado por la curvatura y la regularización.

En el caso de la pérdida cuadrática $\ell(y, \hat{y}) = \frac{1}{2}(y - \hat{y})^2$, se tiene:

$$g_i = \hat{y}_i^{(t-1)} - y_i, \quad h_i = 1.$$

4.6.5 Shrinkage y Submuestreo de Columnas

Además de las técnicas de regularización descritas anteriormente, los autores [6] introducen dos mecanismos adicionales que mejoran la capacidad de generalización del modelo y reducen el riesgo de sobreajuste: el *shrinkage* (también conocido como *learning rate*) y el *column subsampling*.

El término de **shrinkage** actúa como un factor de aprendizaje que escala la contribución de cada nuevo árbol antes de ser agregado al ensamble. En lugar

de actualizar el modelo directamente con la predicción del nuevo árbol $f_t(x)$, la actualización se realiza como:

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + \eta f_t(x_i), \quad (50)$$

donde $\eta \in (0, 1]$ es el parámetro de aprendizaje. Valores pequeños de η reducen la velocidad de aprendizaje, haciendo que el modelo requiera más árboles para converger, pero al mismo tiempo aumentan su capacidad de generalización, evitando sobreajuste prematuro.

Por otro lado, el **submuestreo de columnas** (*column subsampling*) consiste en seleccionar aleatoriamente un subconjunto de las variables predictoras para construir cada árbol. Este método, inspirado en *Random Forests*, ayuda a reducir la correlación entre los árboles y mejora la robustez del modelo, especialmente en presencia de variables altamente correlacionadas. El uso de esta técnica también incrementa la eficiencia computacional, al reducir la cantidad de operaciones necesarias durante el crecimiento de cada árbol.

En conjunto, estas estrategias permiten que XGBoost alcance un mejor equilibrio entre sesgo y varianza, al tiempo que mejora la estabilidad y la eficiencia del proceso de entrenamiento.

4.6.6 Sistema de Diseño

El sistema propuesto en XGBoost fue diseñado para ser escalable tanto en entornos de un solo nodo como en configuraciones distribuidas. Los autores [6] identifican varios cuellos de botella en los algoritmos tradicionales de *gradient boosting* e implementan mejoras que permiten aprovechar al máximo los recursos de hardware disponibles.

4.6.7 Estructuras de datos optimizadas

Una de las principales contribuciones de XGBoost es la utilización de una estructura de datos denominada **block structure**, que almacena las características en formato comprimido y ordenadas de forma precomputada. Esta disposición permite calcular los histogramas de forma eficiente durante la búsqueda de divisiones óptimas en los nodos, reduciendo significativamente el tiempo de entrenamiento.

Además, XGBoost soporta tanto datos densos como dispersos, implementando un manejo explícito de valores faltantes. Los autores muestran que durante el entrenamiento, el modelo aprende automáticamente la dirección (izquierda o derecha) más adecuada para asignar las observaciones con valores faltantes en cada división.

4.6.8 Paralelización del proceso de construcción del árbol

El algoritmo de *tree boosting* en XGBoost permite paralelizar la construcción de histogramas de gradientes y hessianos en diferentes características. Cada núcleo de procesamiento puede calcular de manera independiente las estadísticas de

una o varias variables, para luego combinarlas y determinar la mejor división de nodo. Esto reduce drásticamente el tiempo de entrenamiento sin alterar la exactitud del modelo.

La combinación de las estructuras de datos en bloques con la paralelización por características permite que XGBoost escale de manera casi lineal con el número de núcleos disponibles.

4.6.9 Soporte para aprendizaje distribuido

El sistema también incorpora soporte para aprendizaje distribuido sobre múltiples máquinas. Para ello, XGBoost utiliza un protocolo de comunicación basado en la reducción (*AllReduce*), el cual permite sincronizar los histogramas de gradientes entre los diferentes nodos de cómputo de forma eficiente. Esto hace posible entrenar modelos de gran escala en conjuntos de datos con millones de observaciones y características.

4.6.10 Manejo de memoria y caché

Otra optimización importante en el diseño de XGBoost es la gestión eficiente de la memoria mediante el uso de **cachés por nivel**. Dado que las instancias de datos se reutilizan en múltiples divisiones durante el crecimiento del árbol, mantener sus estadísticas intermedias en memoria permite evitar cálculos redundantes, acelerando significativamente el proceso de entrenamiento.

4.6.11 Soporte para diferentes objetivos y métricas

Finalmente, XGBoost fue diseñado para ser altamente flexible, permitiendo al usuario definir diferentes funciones de pérdida convexas diferenciables ℓ , así como métricas personalizadas de evaluación. Esto permite su aplicación tanto en problemas de regresión como de clasificación binaria y multiclase, e incluso en tareas de ranking y predicción de series temporales.

Estas optimizaciones de diseño, combinadas con las estrategias de regularización y subsampling, convierten a XGBoost en uno de los algoritmos más rápidos, precisos y escalables dentro de la familia de modelos de *gradient boosting*.

Redes Neuronales

Las redes neuronales se basan en el principio del funcionamiento de una neurona, la cual recibe diferentes estímulos, y esta se dispara si se alcanza cierto nivel de “excitación”. En el caso de las redes neurales artificiales, éstas consisten en valores reales x_i como entradas, las cuales son multiplicadas por pesos w_i y éstas son sumadas para finalmente ser procesadas por una función de activación. Esta función por lo general es un sigmoide, pero en la actualidad hay otras que también son usadas, como la tangente hiperbólica. Esto hace que se puedan modelar relaciones no lineales. Aun que en el caso descrito, esto solo comprende una sola neurona. Para entrar más en detalle veamos que...

4.7 Redes Neuronales Recurrentes (RNN)

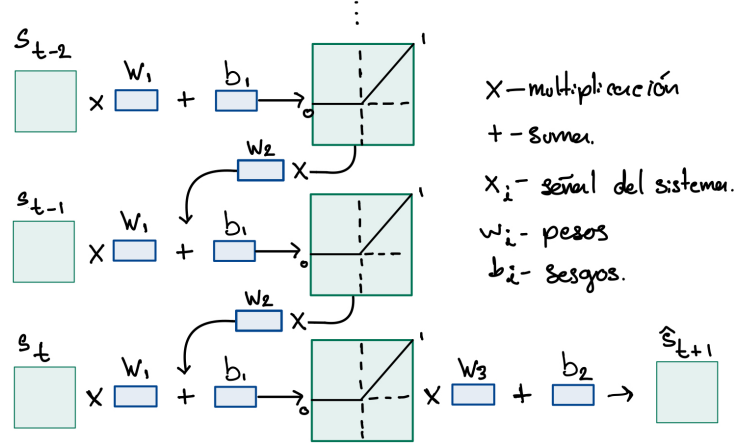


Figura 9: Estructura interna de una neurona recurrente, con funciones de activación ReLU.

Las RNN's tienen como objetivo guardar información sobre todo el estado del sistema, paso a paso, hasta llegar al final de la ventana de n observaciones definidas por el usuario, para generar la predicción $\hat{x}_{t+1}$ del estado futuro o todo un horizonte extendido, dados los n pasos de contexto histórico. La estructura de esto se observa en en la figura 9.

$$s_{t+1} = G\left([s_t \cdot w_1 + G(s'_{t-1}) \cdot w_2 + b_1]\right) \cdot w_3 + b_2 \quad (51)$$

$$s'_{t-1} = s_{t-1} \cdot w_1 + b_1 + G(s'_{t-2}) \cdot w_2. \quad (52)$$

Para entender como surgió está idea, considere la ecuación diferencial ordinaria general, no lineal, de primer grado, no homogénea, que describe la evolución de la señal en un sistema a lo largo del tiempo.

$$\frac{d\vec{s}(t)}{dt} = \vec{f}(t) + \vec{\phi}. \quad (53)$$

$\vec{s}(t)$ es el valor del estado actual de la señal, dado el vector de dimensión d , $\vec{f}(t)$ es una función del tiempo, con $t \in \mathbb{R}^+$, y $\vec{\phi}$ es un vector constante de dimensión d . Esta ecuación se puede ver intuitivamente como la tasa de cambio del agua que hay en una cisterna, a la cual se le agrega agua conforme pasa el tiempo, y al mismo tiempo, está pierde agua durante el proceso; en un enfoque distinto, puede interpretarse como el cambio del precio de una acción.

Una forma de ver a $\vec{f}(t)$ es de la siguiente manera

$$\vec{f}(t) = \vec{h}(\vec{s}(t), \vec{x}(t)). \quad (54)$$

donde $\vec{x}(t)$ es un vector de entrada de dimensión d y $\vec{h}(\vec{s}(t), \vec{x}(t))$ es una función vector-valuada que toma como entrada vectores-valorados. Sustituyendo se tiene

$$\frac{d\vec{s}(t)}{dt} = \vec{h}(\vec{s}(t), \vec{x}(t)) + \vec{\phi}. \quad (55)$$

Nuestro caso de interés, es cuando $f(t)$ es un modelo aditivo de la forma

$$\vec{f}(t) = \vec{a}(t) + \vec{b}(t) + \vec{c}(t) \quad (56)$$

donde nuevamente sus términos son constituidos por funciones del tiempo de dimensión d . Como su nombre lo dice, añade sus términos, no lineales que determinan la tasa de cambio de la actividad neuronal, o sus potenciales $\vec{s}(t)$.

Considere ahora un modelo aditivo saturado, donde sus términos aditivos se definen de la siguiente manera.

$$\vec{a}(t) = \sum_{k=0}^{K_s-1} \vec{a}_k(\vec{s}(t - \tau_s(k))) \quad (57)$$

$$\vec{b}(t) = \sum_{k=0}^{K_r-1} \vec{b}_k(\vec{r}(t - \tau_r(k))) \quad (58)$$

$$\vec{r}(t - \tau_r(k)) = G(\vec{s}(t - \tau_r(k))) \quad (59)$$

$$\vec{c}(t) = \sum_{k=0}^{K_x-1} \vec{c}_k(\vec{x}(t - \tau_x(k))). \quad (60)$$

donde $\vec{r}(t)$ es el vector de la señal $\vec{s}(t)$ procesada por la función de activación $G(z)$, en la figura 9 la función de activación es una ReLU, pero no está limitada a ella.

El sistema resultante es el siguiente

$$\begin{aligned} \frac{d\vec{s}(t)}{dt} = & \sum_{k=0}^{K_s-1} \vec{a}_k(\vec{s}(t - \tau_s(k))) + \sum_{k=0}^{K_r-1} \vec{b}_k(\vec{r}(t - \tau_r(k))) \\ & + \sum_{k=0}^{K_x-1} \vec{c}_k(\vec{x}(t - \tau_x(k))) + \vec{\phi}. \end{aligned} \quad (61)$$

$$\vec{r}(t - \tau_r(k)) = G(\vec{s}(t - \tau_r(k))). \quad (62)$$

El sistema anterior describe la tasa de cambio del estado $\vec{s}(t)$ como la combinación de diferentes efectos rezagados en el tiempo más un termino de sesgo.

Donde, el primer termino $\sum_{k=0}^{K_s-1} \vec{a}_k(\vec{s}(t - \tau_s(k)))$ es el estado rezagado, donde versiones pasadas de este afectan el estado actual, lo que representa la memoria interna del sistema. Por otro lado, el segundo termino $\sum_{k=0}^{K_r-1} \vec{b}_k(\vec{r}(t - \tau_r(k)))$ es similar al primer termino, solo que esta usa la salida transformada por la

función de activación (G), donde $\vec{r}(t)$ es una versión binarizada del estado $\vec{s}(t)$, lo que ayuda a reducir la distorsión. Puede interpretarse a como el perceptrón procesa la información antes de producir la salida. El penúltimo termino, $\sum_{k=0}^{K_x-1} \vec{c}_k(\vec{x}(t - \tau_x(k)))$ incorpora entradas externas rezagadas al sistema, lo que ayuda a modelar el estado $\vec{s}(t)$. Finalmente se tiene el ultimo termino $\vec{\phi}$ el cual representa el sesgo dentro del sistema.

En general, muchos procesos biológicos, físicos, económicos, entre otros; usan frecuentemente valores pasados como predictores, donde la naturaleza de estos procesos trae como consecuencia el uso de ecuaciones rezagadas para ser modelados de forma apropiada. En el caso de las redes neuronales, los rezagos de las variables tanto de respuesta como externas son una parte fundamental del sistema y uno de los factores principales que determinan las dinámicas del sistema.

Los términos rezagados que hemos visto representan la "memoria" del sistema, donde las variables involucradas no dependen del momento actual, sino también de valores pasados, medidos en diferentes momentos. Esto provoca que el sistema tenga en cuenta como ha cambiado el sistema previamente, además de una señal derivada del estado que va acompañada de una forma de la tangente hiperbólica la cual es una transformación no lineal, y además posea señales externas que afectan al sistema, pero evaluadas en instantes previos. Lo que enriquece la capacidad del modelo para capturar la relación causa-efecto y la influencia del contexto temporal, dando como resultado una ecuación con una visión más completa de la historia del sistema y cómo influye en el presente.

Ahora, suponiendo que $\vec{a}$, $\vec{b}$ y $\vec{c}$ son funciones lineales de $\vec{s}$, $\vec{r}$ y $\vec{x}$ respectivamente, entonces la ecuación se convierte en una ecuación diferencial no lineal de diferencias (DDE) con coeficientes lineales.

$$\begin{aligned} \frac{d\vec{s}(t)}{dt} = & \sum_{k=0}^{K_s-1} A_k(\vec{s}(t - \tau_s(k))) \\ & + \sum_{k=0}^{K_r-1} B_k(\vec{r}(t - \tau_r(k))) \\ & + \sum_{k=0}^{K_x-1} C_k(\vec{x}(t - \tau_x(k))) + \vec{\phi}. \end{aligned} \quad (63)$$

Al aplicar las siguientes condiciones $K_s = 1$, $\tau_s(0) = 0$, $A_0 = A$, $K_r = 1$, $\tau_r(0) = \tau_0$, $B_0 = B$, $K_x = 1$, $\tau_x(0) = 0$, $C_0 = C$ a la ecuación anterior se tiene lo siguiente

$$\frac{d\vec{s}(t)}{dt} = A\vec{s}(t) + B\vec{r}(t - \tau_0) + C\vec{x}(t) + \vec{\phi}. \quad (64)$$

Estas ecuaciones son no-lineales de primer orden no-homogéneas. Una forma de evaluar estas ecuaciones es discretizar las en tiempo y calcular los valores de las

señales de entrada y las señales de estado en cada momento sampleado hasta la duración total que se requiera, esto implica integración numérica. [25]

Sea ΔT la duración total y n el índice del tiempo en la aplicación de la regla discreta inversa de Euler aplicada a la ecuación anterior, por lo tanto

$$t = n\Delta T \quad (65)$$

$$\frac{d\vec{s}(t)}{dt} \approx \frac{\vec{s}(n\Delta T + \Delta T) - \vec{s}(n\Delta T)}{\Delta T} \quad (66)$$

$$A\vec{s}(t) + B\vec{r}(t - \tau_0) + C\vec{x}(t) + \vec{\phi} = A\vec{s}(n\Delta T) + B\vec{r}(n\Delta T - \tau_0) + C\vec{x}(n\Delta T) + \vec{\phi} \quad (67)$$

$$\frac{\vec{s}(n\Delta T + \Delta T) - \vec{s}(n\Delta T)}{\Delta T} \approx A\vec{s}(n\Delta T + \Delta T) + B\vec{r}(n\Delta T + \Delta T - \tau_0) + C\vec{x}(n\Delta T + \Delta T) + \vec{\phi}. \quad (68)$$

El valor de rezago τ_0 , sé iguala a un valor fijo, la razón de esto, es por que al avanzar, el contenido en la memoria se sobre escribe con un valor actualizado de la señal procesada, el cual se usará en el siguiente paso de tiempo, y así sucesivamente. Es por esto que se establece $\tau_0 = \Delta T$ y se reemplaza el signo de aproximación por el igual, de está manera se tiene...

$$\frac{\vec{s}(n\Delta T + \Delta T) - \vec{s}(n\Delta T)}{\Delta T} = A\vec{s}(n\Delta T + \Delta T) + B\vec{r}(n\Delta T) + C\vec{x}(n\Delta T + \Delta T) + \vec{\phi} \quad (69)$$

$$\frac{\vec{s}((n+1)\Delta T) - \vec{s}(n\Delta T)}{\Delta T} = A\vec{s}((n+1)\Delta T) + B\vec{r}(n\Delta T) + C\vec{x}((n+1)\Delta T) + \vec{\phi} \quad (70)$$

$$\vec{s}((n+1)\Delta T) - \vec{s}(n\Delta T) = \Delta T \left(A\vec{s}((n+1)\Delta T) + B\vec{r}(n\Delta T) + C\vec{x}((n+1)\Delta T) + \vec{\phi} \right). \quad (71)$$

Al aplicar discretización, todas las medidas de tiempo en la ecuaciones anterior se convierten en múltiplos integrales del paso de tiempo sampleando, ΔT . Al eliminar ΔT de los argumentos, queda el eje de tiempo sin dimensión. Por lo tanto, todas las señales son transformadas en secuencias, cuyo dominio es el índice discreto n , y la ecuación 14 se convierte en una ecuación diferencial, no lineal, de primer orden y además, no homogénea. [25]

$$\vec{s}[n+1] - \vec{s}[n] = \Delta T \left(A\vec{s}[n+1] + B\vec{r}[n] + C\vec{x}[n+1] + \vec{\phi} \right) \quad (72)$$

$$\vec{s}[n+1] = \vec{s}[n] + \Delta T \left(A\vec{s}[n+1] + B\vec{r}[n] + C\vec{x}[n+1] + \vec{\phi} \right) \quad (73)$$

$$\vec{s}[n+1] = \vec{s}[n] + \Delta T \cdot A\vec{s}[n+1] + \Delta T \cdot B\vec{r}[n] + \Delta T \cdot C\vec{x}[n+1] + \Delta T \cdot \vec{\phi} \quad (74)$$

$$\vec{s}[n+1] - \Delta T \cdot A\vec{s}[n+1] = \vec{s}[n] + \Delta T \cdot B\vec{r}[n] + \Delta T \cdot C\vec{x}[n+1] + \Delta T \cdot \vec{\phi} \quad (75)$$

$$(I - \Delta T \cdot A)\vec{s}[n+1] = \vec{s}[n] + \Delta T \cdot B\vec{r}[n] + \Delta T \cdot C\vec{x}[n+1] + \Delta T \cdot \vec{\phi}. \quad (76)$$

Sea

$$W_s = (I - \Delta T \cdot A)^{-1}, \quad (77)$$

multiplicando la ecuación pasada por W_s da como resultado

$$\vec{s}[n+1] = W_s\vec{s}[n] + \Delta T \cdot W_s B\vec{r}[n] + \Delta T \cdot W_s C\vec{x}[n+1] + \Delta T \cdot W_s \vec{\phi}. \quad (78)$$

restando uno al indice, la ecuación se transforma a

$$\vec{s}[n] = W_s \vec{s}[n-1] + \Delta T \cdot W_s B \vec{r}[n-1] + \Delta T \cdot W_s C \vec{x}[n] + \Delta T \cdot W_s \vec{\phi} \quad (79)$$

$$\vec{r}[n] = G(\vec{s}[n]). \quad (80)$$

Ahora, se definen las siguientes matrices de pesos y vectores de sesgo,

$$W_r = (\Delta T)B \quad (81)$$

$$W_x = (\Delta T)C \quad (82)$$

$$\vec{\theta}_s = (\Delta T)W_s \vec{\phi}. \quad (83)$$

los cuales finalmente transforman el sistema en la forma canónica de la Red Neuronal Recurrente (RNN)

$$\vec{s}[n] = W_s \vec{s}[n-1] + W_r \vec{r}[n-1] + W_x \vec{x}[n] + \vec{\theta}_s \quad (84)$$

$$\vec{r}[n] = G(\vec{s}[n]). \quad (85)$$

En la ecuación 84, note que la señal procesada por la función de activación vuelve alimentar a la célula en la siguiente observación, y esto ocurre varias veces, como se observa en la figura 9, lo que provoca que el gradiente se vuelva demasiado grande y este explote, o que de plano se de la desaparición forzada de este.

[25] Escribe en su proposición 1, que una sola célula de la RNN, extendida por un numero finito de pasos, es computacionalmente capaz de encontrar a Θ tal que optimice L sobre el conjunto de entrenamiento y que ademas pueda hacer inferencias sobre valores no vistos.

En general, surgen varios problemas con este sistema, el primero es que entre más observaciones existan en las secuencias de entrenamiento, la probabilidad que se obtengan buenos resultados aumenta, pero esto implica que aumentará la cantidad de cálculos computacionales que se tienen que realizar.

Por otra parte, si un sistema recurrente se extiende por una cierta cantidad de pasos pequeños, y es usado para predecir una secuencia de datos que exhibe tendencias de largo plazo, el sistema será entrenado bajo las hipótesis incorrectas, ya que este espera que los resultados de salida sean secuencias del mismo tamaño, donde en este caso es corto, por lo que la rnn no podrá capturar dependencias de largo plazo.

Uno de los problemas de las RNN's es que tras extenderlas a lo largo de varias observaciones, éstas no logran pasar más allá de 5 a 10 pasos de acuerdo con [25]. Esto se debe al hecho de que al entrenar esta red neuronal, los gradientes tienden a crecer o disminuir con cada observación. Por lo que al transcurrir varias observaciones el gradiente explota o este se desvanece. El análisis detallado puede verse en [25]

4.8 Red Neuronal Recurrente con Memoria de Largo y Corto Plazo (LSTM)

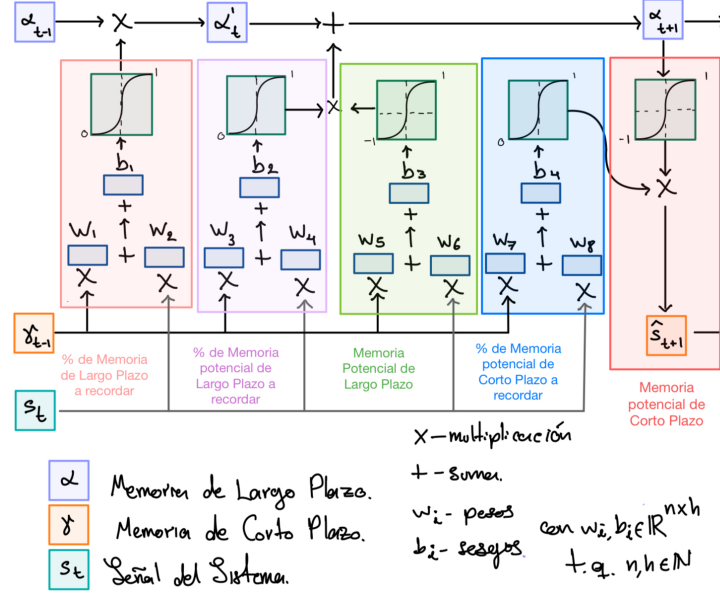


Figura 10: Estructura interna de una neurona recurrente

Las LSTMs surgen como la solución al problema de la desaparición y explosión de gradiente que ocurre cuando se entrena una RNN. Para ello se implemento un mecanismo que guarda dependencias temporales de corto como de largo plazo, de una forma no lineal dentro de la célula de la RNN, esto con la finalidad de que el gradiente de la función objetivo de la señal a modelar no se desvanezca o explote como ya mencionamos, véase la figura 10.

El primer paso es separar la parte derecha de la ecuación 30.

$$\vec{s}[n] = \vec{F}_s(\vec{s}[n-1]) + \vec{F}_u(\vec{r}[n-1], \vec{x}[n]) \quad (86)$$

$$\vec{r}[n] = G_d(\vec{s}[n]) \quad (87)$$

$$\vec{F}_s(\vec{s}[n-1]) = W_s \vec{s}[n-1] \quad (88)$$

$$\vec{F}_u(\vec{r}[n-1], \vec{x}[n]) = W_r \vec{r}[n-1] + W_x \vec{x}[n] + \vec{\theta}_s. \quad (89)$$

donde nuevamente la función $G_d(\vec{z})$ es la tangente hiperbólica. Notese que en la ecuación 34, el estado de la señal se combina de forma igualitaria en cada observación, pero esto se puede ajustar al multiplicar ambas cantidades por "puertas especiales", $\vec{g}_{cs}[n]$ control del estado $\vec{g}_{cu}[n]$ control de actualización-

espectivamente.

$$\vec{s}[n] = \vec{g}_{cs}[n] \odot \vec{F}_s(\vec{s}[n-1]) + \vec{g}_{cu}[n] \odot \vec{F}_u(\vec{r}[n-1], \vec{x}[n]) \quad (90)$$

$$\vec{0} \leq \vec{g}_{cs}[n], \vec{g}_{cu}[n] \leq \vec{1}. \quad (91)$$

Los elementos de las señales de las puertas son fracciones no negativas como se puede ver en la desigualdad de la ecuación 39. Por otro lado, la función $\vec{g}_{cs}[n]$ se encarga de controlar la cantidad de información del estado de la señal rezagado de la observación anterior, mientras que $\vec{g}_{cu}[n]$ regula la cantidad de información de la señal actualizada a ser inyectada en la observación actual. Finalmente, como W_s es una matriz diagonal con entradas positivas fraccionarias $\frac{1}{|a_{ii}|}$ y dado que los elementos en $\vec{g}_{cs}[n]$ son también fracciones, se puede establecer a $W_s = I$ siempre y cuando las funciones de "puerta" sean paramétricas y sus parámetros sean aprendidos durante el entrenamiento, por lo que se puede simplificar la ecuación 36 a $F_s(\vec{s}[n-1]) = \vec{s}[n-1]$

Véase ahora el termino de actualización $\vec{F}_u(\vec{r}[n-1], \vec{x}[n])$, el cuál es controlado por $\vec{g}_{cu}[n]$, nótese que en la ecuación 37, la función sigmoideal aplicada a la observación pasada del estado de la señal y la señal de la observación actual constituyen el candidato de actualización en cada observación con índice n , donde ambos términos contribuyen a ella en la misma proporción. El problema surge cuando $\vec{g}_{cu}[n] \sim 1$, ya que $\vec{s}[n-1]$ y $\vec{s}[n]$ se conectan a través de W_r y la función sigmoideal $Gd.$ está problemática limita al gradiente en la función objetivo respecto al estado de la señal, lo que trae como consecuencia al sistema la explosión o la desaparición forzada del gradiente, por lo que la función $\vec{r}[n]$ será controlada por otra puerta llamada "puerta de control" $\vec{g}_{cr}[n]$ de la siguiente manera

$$\vec{v}[n] = \vec{g}_{cr}[n] \odot \vec{r}[n] \quad (92)$$

$$\vec{0} \leq \vec{g}_{cr}[n] \leq \vec{1}. \quad (93)$$

La puerta de control $\vec{g}_{cr}[n]$, determina el porcentaje de información de la función sigmoideal aplicado al estado de la observación pasada que será usado para la observación actual del estado n . Por lo que reemplazando $\vec{v}[n-1]$ por $\vec{r}[n-1]$ en la ecuación 37, está se transforma en

$$\vec{F}_u(\vec{v}[n-1], \vec{x}[n]) = W_r \vec{v}[n-1] + W_x \vec{x}[n] + \vec{\theta}_s. \quad (94)$$

Véase ahora que aun que la entrada externa $\vec{x}[n]$ no afecta a la estabilidad del sistema o contribuye al problema de la explosión del gradiente o su desaparición forzada, emparejar está entrada con otra puerta de control $\vec{g}_{cx}[n]$ única para si misma, ayuda al sistema a ser más flexible. Por lo que al incorporar esto en la ecuación anterior...

$$\vec{F}_u(\vec{v}[n-1], \vec{x}[n]) = W_r \vec{v}[n-1] + \vec{g}_{cx}[n] \odot W_x \vec{x}[n] + \vec{\theta}_s. \quad (95)$$

Extendiendo por completo el sistema

$$\vec{s}[n] = \vec{g}_{cs}[n] \odot \vec{s}[n-1] + \vec{g}_{cu}[n] \odot [W_r \vec{v}[n-1] + \vec{g}_{cx}[n] \odot W_x \vec{x}[n] + \vec{\theta}_s]. \quad (96)$$

Ahora, para hacer más robusto el sistema, resiliente y estable, se aplica una función no lineal de tipo tangente hiperbólica $G_d(z)$ a la señal agregada $\vec{F}_u(\vec{v}[n-1], \vec{x}[n])$ lo que permite comprimir los valores y que estos se mantengan dentro del rango producido por la célula $\vec{v}[n]$ y la contribución de la señal de entrada $\vec{g}_{cx}[n] \odot \vec{x}_n$, el termino de actualización $\vec{u}[n]$ queda...

$$\vec{u}[n] = G_d(F_u(\vec{v}[n-1], \vec{x}[n])). \quad (97)$$

Finalmente, sustituyendo el termino de actualización en la ecuación 44

$$\vec{s}[n] = \vec{g}_{cs}[n] \odot \vec{s}[n-1] + \vec{g}_{cu}[n] \odot \vec{u}[n]. \quad (98)$$

La ecuación anterior es la culminación de todo el sistema de formulas que definen la red neuronal LSTM.

Los términos relacionados a las puertas de las señales $\vec{g}_{cs}[n]$, $\vec{g}_{cr}[n]$ y $\vec{g}_{cu}[n]$ son en realidad funciones sigmoidal-es y tangentes hiperbólicas, cuyas expresiones matemáticas son las siguientes.

$$\text{Sigmoide: } f(x) = \frac{e^x}{e^x + 1} \quad (99)$$

$$\text{Hiperbólica Tangente: } f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad (100)$$

Notese que estás funciones son diferenciables, monótonas crecientes, no lineales, continuas en todo su dominio $(-\infty, \infty)$ y cuyo rango comprende $(0, 1)$ y $(-1, +1)$ respectivamente.

Véase ahora, que los términos que alimentaran las funciones de cada "puerta" son los siguientes

$$\vec{z}_{cs}[n] = W_{x_{cs}} \vec{x}[n] + W_{s_{cs}} \vec{s}[n-1] + W_{v_{cs}} \vec{v}[n-1] + \vec{\theta}_{cs} \quad (101)$$

$$\vec{z}_{cu}[n] = W_{x_{cu}} \vec{x}[n] + W_{s_{cu}} \vec{s}[n-1] + W_{v_{cu}} \vec{v}[n-1] + \vec{\theta}_{cu} \quad (102)$$

$$\vec{z}_{cr}[n] = W_{x_{cr}} \vec{x}[n] + W_{s_{cr}} \vec{s}[n-1] + W_{v_{cr}} \vec{v}[n-1] + \vec{\theta}_{cr} \quad (103)$$

Note que los valores que se emplean son todas las señales disponibles $\vec{s}[n-1]$, $\vec{v}[n-1]$, $\vec{x}[n]$, $\vec{s}[n]$, $\vec{v}[n-1]$, $\vec{x}[n]$ cercanas a la observación actual n .

Por lo tanto, las funciones de "puerta" quedan definidas como

$$\vec{g}_{cs}[n] = G_c(\vec{z}_{cs}[n]) \quad (104)$$

$$\vec{g}_{cu}[n] = G_c(\vec{z}_{cu}[n]) \quad (105)$$

$$\vec{g}_{cr}[n] = G_c(\vec{z}_{cr}[n]) \quad (106)$$

$$(107)$$

Esto hace que durante el entrenamiento y la inferencia, las puertas se vayan moldeando correspondientemente a las señales, al utilizar toda la información disponible en cada observación. Esto hace que las puertas detecten y mitiguen anomalías en las secuencias de entrada; haciendo el sistema de la lstm un modelo robusto que compensa las imperfecciones en los datos y capaz de generar secuencias de salida realmente buenas. [1]

4.9 DeepAr

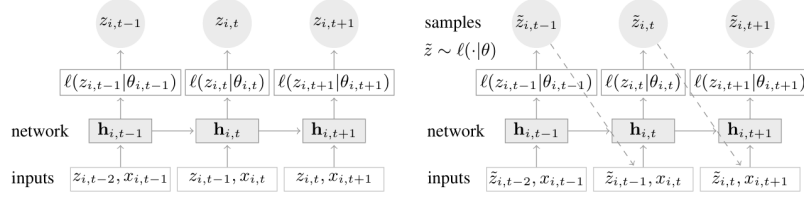


Figura 11: Estructura de DeepAr, imagen de [24]

DeepAr es un modelo desarrollado por “Amazon”[24]. DeepAr es un método basado en redes neuronales recurrentes autoregresivas que aprenden toda la historia de todas las series temporales en el conjunto de datos que se tenga, el cual utiliza LSTM’s y una arquitectura similar al problema de predicción probabilístico. De acuerdo con los autores, DeepAr se luce al resolver el problema que se presenta al tratar de modelar múltiples series temporales, las cuales tienen diferentes características, principalmente, la continuidad y la escala de las mismas, así como también nuevas series temporales con pocos datos dada su incorporación reciente (Por ejemplo, la entrada de un producto al mercado, y que por consecuencia no se tienen información histórica de este.)

Tomando la notación de [24], se denota el valor de una serie temporal con i en el tiempo t por $z_{i,t}$. El objetivo principal es modelar la distribución condicional de cada serie temporal que tendrá en el futuro $[z_{i,t_0}, z_{i,t_0+1}, \dots, z_{i,T}] := \mathbf{z}_{i,t_0:T}$, dado su pasado $[z_{i,1}, \dots, z_{i,t_0-2}, z_{i,t_0-1}] := \mathbf{z}_{i,1:t_0-1}$.

$$P(\mathbf{z}_{i,t_0:T} | \mathbf{z}_{i,1:t_0-1}, \mathbf{x}_{i,1:T}) \quad (108)$$

Donde t_0 denota la primer observación desconocida al momento de realizar la predicción, mientras que $\mathbf{x}_{i,1:T}$ representa las variables conocidas en el futuro que explican la variable a ser predicha.

Como se menciona, el modelo se basa en la noción de probabilidad condicional, la cual indica que tan probable es un evento dada una condición.

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

Esta distribución representa todas las posibles variantes futuras del sistema, ponderadas por su probabilidad de ocurrencia.

DeepAr se basa en una arquitectura auto-regresiva recurrente, en el sentido de que al predecir un valor $z_{i,t-1}$, toma ese mismo valor para alimentarse y obtener la siguiente predicción y al mismo tiempo toma el valor pasado de la salida de red $h_{i,t-1}$, tal como se vio con la RNN y la LSTM. Por otro lado asume que la distribución condicional de los valores futuros de las series temporales, consiste en el producto de distribuciones condicionadas, parametrizadas por la

salida de la red $\mathbf{h}_{i,t}$.

$$Q_{\Theta}(\mathbf{z}_{i,t_0:T}|\mathbf{z}_{i,1:t_0-1}, \mathbf{x}_{i,1:T}) = \prod_{t=t_0}^T Q_{\Theta}(z_{i,t}|\mathbf{z}_{i,t:t-1}, \mathbf{x}_{i,1:T}) = \prod_{t=t_0}^T l(z_{i,t}|\theta(\mathbf{h}_{i,t}, \Theta)) \quad (109)$$

Donde

$$\mathbf{h}_{i,t} = h(\mathbf{h}_{i,t-1}, z_{i,t-1}, x_{i,t}, \Theta) \quad (110)$$

hace referencia a la salida de una red neuronal lstm. En general el modelo admite n capas LSTM con m células por capa, con $n, m \in \mathbb{N}$. La salida de la red neuronal sirve para crear los parámetros de la distribución $l(z_{i,t}|\theta(\mathbf{h}_{i,t}))$, a partir de la cual se infiere el valor futuro $z_{i,t}$.

Para ello se aplican dos transformaciones a la salida o estado oculto generado por la red $\mathbf{h}_{i,t}$. Para el caso de una distribución normal, se predice la media y la desviación estándar de la siguiente manera

$$l_G(z|\mu, \sigma) = (2\pi\sigma^2)^{-\frac{1}{2}} \exp\left(-\frac{(z-\mu)^2}{2\sigma^2}\right) \quad (111)$$

$$\mu(\mathbf{h}_{i,t}) = \mathbf{w}_{\mu}^T \mathbf{h}_{i,t} + b_{\mu} \quad (112)$$

$$\sigma(\mathbf{h}_{i,t}) = \log(1 + \exp(\mathbf{w}_{\sigma}^T \mathbf{h}_{i,t} + b_{\sigma})) \quad (113)$$

Mientras que para una distribución binomial negativa, está se parametriza por su media $\mu \in \mathbb{R}^+$ y el parámetro $\alpha \in \mathbb{R}^+$, permitiendo que de esta manera la varianza dependa de la media $\text{Var}(z) = \mu + \alpha\mu^2$.

$$l_{NB}(z|\mu, \sigma) = \frac{\Gamma(z + \frac{1}{\alpha})}{\Gamma(z+1)\Gamma(\frac{1}{\alpha})} \left(\frac{1}{1 + \alpha\mu}\right)^{\frac{1}{\alpha}} \left(\frac{\alpha\mu}{1 + \alpha\mu}\right)^z \quad (114)$$

$$\mu(\mathbf{h}_{i,t}) = \log(1 + \exp(\mathbf{w}_{\mu}^T \mathbf{h}_{i,t} + b_{\mu})) \quad (115)$$

$$\sigma(\mathbf{h}_{i,t}) = \log(1 + \exp(\mathbf{w}_{\sigma}^T \mathbf{h}_{i,t} + b_{\sigma})) \quad (116)$$

$$(117)$$

En la practica se pueden usar diferentes distribuciones dependiendo el conjunto de datos y la distribución de estos, siempre y cuando sea fácil generar muestras de estas distribuciones, se pueda calcular el logaritmo de la función de verosimilitud con respecto a los parámetros, y los gradientes puedan ser evaluados con respecto a los parámetros del modelo para poder ser optimizados. En el caso de series temporales con valores positivos se suele usar la distribución binomial negativa.

Para entrenar el modelo, se parte de un conjunto de series temporales $\{\mathbf{z}_{i,1:T}\}_{i=1,\dots,N}$ junto con sus respectivas variables predictoras $\{\mathbf{x}_{i,1:T}\}_{i=1,\dots,N}$. Se escoge un intervalo de tiempo en el que los valores $z_{i,t}$ dentro del horizonte de predicción sean conocidos, permitiendo calcular la función de pérdida basada en la verosimilitud.

Los parámetros del modelo, que incluyen tanto los pesos de la red neuronal recurrente $h(\cdot)$ como los parámetros de la función $\theta(\cdot)$ que mapea el estado oculto $h_{i,t}$ a los parámetros de la distribución de salida, pueden ser aprendidos mediante el algoritmo del descenso estocástico del gradiente. La función objetivo a maximizar es la log-verosimilitud total de las observaciones:

$$\mathcal{L} = \sum_{i=1}^N \sum_{t=t_0}^T \log \ell(z_{i,t} \mid \theta(\mathbf{h}_{i,t})) \quad (118)$$

Dado que $h_{i,t}$ es una función determinista de las entradas, no se requiere inferencia sobre variables latentes, lo cual permite optimizar directamente la función de log-verosimilitud a través de gradientes computables con respecto a todos los parámetros del modelo.

En la práctica, durante el entrenamiento, correspondiente al intervalo $t = 1, \dots, t_0 - 1$, se va alimentando a DeepAr con las observaciones reales $z_{i,t}$ junto con las variables predictoras $x_{i,t}$ al modelo, el cual actualiza su estado oculto $\mathbf{h}_{i,t}$ a través de una red lstm compartida. El último estado oculto h_{i,t_0-1} toma la información relevante a toda la historia observada y sirve como un vector de contexto. A partir de ese estado inicial, comienza el proceso auto-regresivo, es decir, la generación de valores desconocidos, para el tiempo $t = t_0, \dots, T$, en donde DeepAr utiliza las predicciones pasadas $\hat{z}_{i,t}$ como entrada, junto con las variables predictoras $\mathbf{x}_{i,t}$. Cabe recalcar que los parámetros de la LSTM son fijos y se comparten en el proceso general.

4.10 Transformer

El modelo Transformer, introducido por Vaswani et al. (2017), revolucionó el procesamiento de secuencias al eliminar por completo el uso de redes recurrentes. Su principal innovación es el mecanismo de atención, que permite capturar dependencias de largo alcance en las secuencias de entrada sin necesidad de procesarlas secuencialmente. Esta arquitectura ha sido la base para múltiples avances en modelado secuencial, incluyendo su adaptación a series temporales como es el caso del modelo Informer.

El modelo fue diseñado originalmente con el objetivo de traducir oraciones a otro idioma. Su funcionamiento es totalmente distinto al acercamiento que se tiene con las redes neuronales recurrentes, o el caso probabilístico de DeepAr. Por lo que vale la pena incorporarlo a este experimento/investigación.

La arquitectura de transformer se divide en dos partes, el codificador y decodificador, para luego unirse; como se muestra en la siguiente figura.

4.10.1 Codificador

La parte izquierda hace referencia al codificador, este se divide en 4 partes, la primer parte consiste en la transformación de palabras a números, ya que, una red neuronal solo procesa números. En el artículo *Attention Is All You Need*

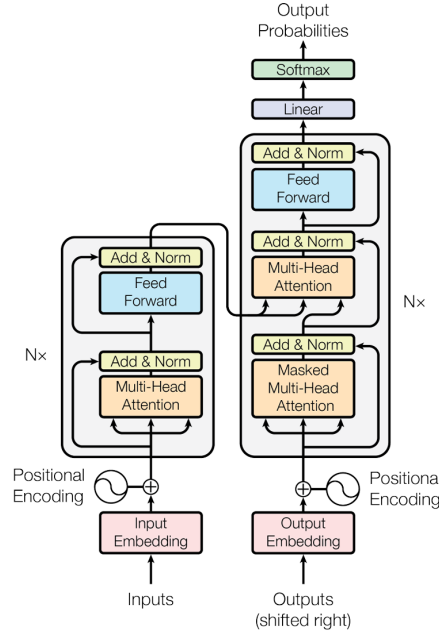


Figura 12: Arquitectura general del modelo Transformer, adaptado de Vaswani et al. (2017) [28].

establecen la dimensión del modelo como $d_{\text{model}} = 512$, esto quiere decir que cada palabra se representará como un vector de dimensión 512 antes de poder ser procesado por las capas de atención.

4.10.2 Representación vectorial - Embeddings

Como se mencionó antes, dado que una red neuronal trabaja con números, cada palabra en el vocabulario definido en el entrenamiento, es representada de forma única por una matriz $W \in \mathbb{R}^{V \times d_{\text{model}}}$, donde V corresponde a la cantidad única de palabras/“tokens” con las que el modelo fue entrenado. En [1] hay un total de 32,000 tokens para el problema de traducción de Inglés - Alemán. Es importante mencionar, que en este aspecto, tanto el codificador como el decodificador poseen distintos vocabularios, por lo que cada vocabulario es codificado a su manera, con su respectiva matriz de pesos $W^{V_1} \in \mathbb{R}^{V_1 \times d_{\text{model}}}$ y $W^{V_2} \in \mathbb{R}^{V_2 \times d_{\text{model}}}$. Por otro lado, es importante mencionar que el vocabulario contiene tokens especiales.

$\langle \text{PAD} \rangle \rightarrow \text{Relleno}$
 $\langle \text{START} \rangle \rightarrow \text{Inicio de la secuencia}$
 $\langle \text{EOS} \rangle \rightarrow \text{Fin de la secuencia}$
 $\langle \text{UNK} \rangle \rightarrow \text{Desconocido}$

Cuando se entrena el modelo, se construye un vocabulario, donde se ordena

Tokenización	
Índice	Token
0	<PAD>
1	<START>
2	Carolina
3	Sur
4	Omelet
5	come
$\vdots$	$\vdots$
V	<EOS>

Tabla 1: Ejemplo de la lista ordenada de todos los tokens (palabras) introducidos al modelo

una lista de tokens (“palabras o sub-palabras”) para que el modelo pueda procesarlos, como se muestra en la tabla 1. De esta manera, cada palabra en una secuencia (una oración) es primero transformada en un número entero mediante un proceso de tokenización, basado en un vocabulario fijo de tamaño V . Este vocabulario asigna un índice único a cada palabra o subpalabra conocida por el modelo.

Véase este caso práctico, dada la oración “Carolina come un omelet”, esta secuencia se divide, lo que da como resultado una secuencia de 4 palabras.

[“Carolina”, “come”, “un”, “omele”]

Dentro de nuestro ejemplo, “Carolina” corresponde al índice 2 en el tokenizador, por lo tanto, su representación inicial será un vector $\mathbf{x}_{Carolina} = [0, 0, 1, \dots, 0] \in \mathbb{R}^V$. Luego, este será multiplicado por la matriz de embeddings $W \in \mathbb{R}^{V \times d_{model}}$ correspondiente al codificador. Finalmente, se obtiene el embedding final $\mathbf{e}_{carolina} = \mathbf{x}_{Carolina} \cdot W$. El proceso es análogo para el resto de palabras.

$$\mathbf{e}_i = \mathbf{x}_i \cdot W \quad (119)$$

4.10.3 Codificación Posicional

Ahora, es importante el orden en el que se van presentando las palabras, ya que no es lo mismo decir “Carolina come un omelet” a decir, “Un omelet come Carolina”, ambas oraciones tienen las mismas palabras, pero ambas tienen un significado completamente distinto y además, se pierde el sentido en la última. La solución que se propone en [28] es utilizar una codificación posicional basada en funciones sinusoidales. Por lo que a cada posición pos en la secuencia de palabras se le asocia un vector $\mathbf{p}_{pos} \in \mathbb{R}^d$, al cual se le suma el embedding de la palabra correspondiente

$$\text{PE}_{(pos,2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d}}}\right), \quad \text{PE}_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d}}}\right) \quad (120)$$

donde pos es la posición de la palabra en la secuencia $(0, 1, 2, \dots)$, i hace referencia a la dimensión en el vector del embedding y d es la dimensión del vector. De esta manera

$$\tilde{\mathbf{e}}_i = \mathbf{e}_i + \mathbf{p}_i \quad (121)$$

donde $e_i \in \mathbb{R}^{d_{model}}$ es el embedding de la palabra, $\mathbf{p}_i \in \mathbb{R}^{d_{model}}$ es el vector de codificación posicional dada la posición i . La ventaja de usar funciones de seno y coseno es que son cíclicas, además permiten capturar las diferencias relativas de acuerdo a su posición entre los elementos de la secuencia, y al mismo tiempo, se evita la necesidad de introducir más parámetros.

Si extendemos nuestra oración inicial a “Carolina come un omelet con sur en un lindo lugar” y aplicamos este principio a nuestra oración estableciendo la dimensión del modelo a 4, las funciones de seno y coseno se verían de la siguiente manera como se muestra en la Figura 13.

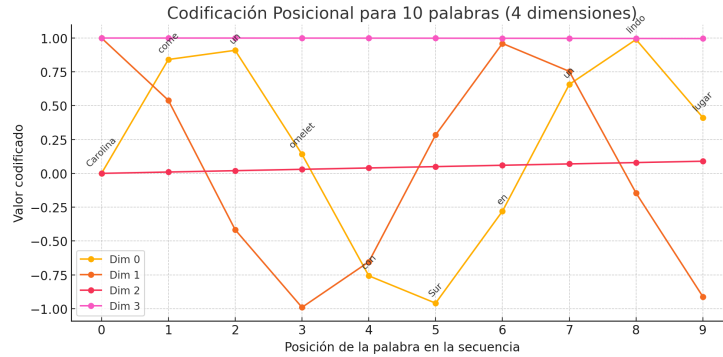


Figura 13: Codificación posicional para la oración “*Carolina come un omelet con Sur en un lindo lugar*” con $d = 4$. Cada dimensión representa una componente del vector posicional asociado a cada palabra, y varía según su posición en la secuencia.

4.10.4 Mecanismo de Atención

Una vez que se tiene el posicionamiento codificado de cada palabra $\tilde{\mathbf{e}}_i \in \mathbb{R}^{d_{model}}$, a cada una de ellas se le aplican tres transformaciones lineales con la finalidad de que cada una proporcione “luz” al modelo. Es por ello que en el mecanismo de atención que se define en [28] existen consultas Q_i (“Queries”), llaves K_i

(“Keys”) y valores V_i (“Values”) cada uno con una finalidad distinta.

$$Q_i = \tilde{\mathbf{e}}_i W^Q \quad (122)$$

$$K_i = \tilde{\mathbf{e}}_i W^K \quad (123)$$

$$V_i = \tilde{\mathbf{e}}_i W^V \quad (124)$$

donde $W^Q, W^K, W^V \in \mathbb{R}^{d_{model} \times d_{model}}$, las consultas se encargan de resolver la pregunta “¿Qué estoy buscando?”, mientras que las llaves “¿Qué tengo para ofrecer?” y el valor “¿Qué puedo transmitir si soy relevante?”.

Para que el modelo pueda saber como se relacionan las palabras entre si, los autores [28] calculan la similitud (“score”) al hacer el producto punto entre la consulta de una palabra Q_i contra todas las llaves K_j , incluida su propia llave.

$$\text{score}_{i,j} = Q_i \cdot K_j \quad (125)$$

De está manera se mide la afinidad o relevancia que hay entre la palabra en la posición i y la palabra en j . Por otro lado, con la finalidad de que los valores no se vuelvan demasiado grandes y se sature la función exponencial normalizada (“softmax”), se escalan los valores al dividirlos entre la raíz cuadrada de d_k .

$$\text{score escalado}_{i,j} = \frac{Q_i \cdot K_j}{\sqrt{d_k}} \quad (126)$$

Luego, se aplica la función softmax a todas las puntuaciones j para una posición i .

$$\alpha_{i,j} = \text{softmax}\left(\frac{Q_i \cdot K_j}{\sqrt{d_k}}\right) \quad (127)$$

De esta manera se generan pesos de atención, es decir, una distribución de probabilidad que le permite al modelo saber a qué otras palabras se les debe de “atender”. Finalmente, la salida de atención para la palabra i se calcula como el promedio ponderado de los valores V_j .

$$\text{Atención}_i = \sum_j \alpha_{i,j} V_j \quad (128)$$

De esta forma, cada palabra tiene una nueva representación cuya dependencia se basa en el contexto de toda la secuencia. Este proceso se hace para todas las consultas de manera simultanea dentro de una matriz Q , así como las llaves K y los valores V , por lo que de forma general se tiene

$$\text{Atención}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (129)$$

Esto conforma una sola función de atención con llaves, valores y consultas, donde $d_k = 512 = d_{model}$.

4.10.5 Atención de Cabeza Múltiple - Multi-Head Attention

Los autores [28] encontraron que en realidad es mucho mejor hacer h proyecciones lineales con diferentes pesos en cada proyección con dimensiones $d_k = d_v = d_{\text{model}}/h = 64$, con $h = 8$ en el artículo de [28] donde h representa la cantidad de cabezas de atención paralelas, cada una aprendiendo diferentes patrones en la secuencia. De esta manera es posible que secuencias de datos más largas y complejas puedan relacionarse mejor.

$$\text{Multi-Atención}(Q, K, V) = \text{Concatenación}(\text{cabeza}_1, \dots, \text{cabeza}_h)W^O \quad (130)$$

donde cada $\text{cabeza}_i = \text{Atención}(QW_i^Q, KW_i^K, VW_i^V)$ y las proyecciones W_i^Q , W_i^K , W_i^V son matrices de pesos específicas para cada cabeza. Las salidas de cada cabeza son concatenadas y transformadas por una última matriz de pesos $W^O \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$ que proyecta el resultado nuevamente al espacio original del modelo. Este mecanismo permite que el modelo atienda simultáneamente a distintas partes de la secuencia desde diferentes perspectivas, lo cual es clave para capturar relaciones semánticas complejas en estructuras de lenguaje o datos temporales.

Tras aplicar la Atención Multi-Cabeza, se tiene una salida por cada palabra, que se denotará por

$$\text{MHA}(\tilde{\mathbf{e}}_i)$$

el cual es un vector de dimensión d_{model} .

4.10.6 Conexiones Residuales

La conexión residual consiste en sumar esta salida con la entrada original, es decir, el vector codificado posicionalmente $\tilde{\mathbf{e}}$ con $\text{MHA}(\tilde{\mathbf{e}}_i)$. Esto se ve de la siguiente manera.

$$\mathbf{y}_i = \tilde{\mathbf{e}}_i + \text{MHA}(\tilde{\mathbf{e}}_i) \quad (131)$$

En pocas palabras, el nuevo vector es el resultado de aplicar atención, sumando lo que ya se sabía. En general, una conexión residual ayuda a preservar la información original, evitando que se pierda lo que ya se sabe. Note que la conexión residual no bloquea el aprendizaje, pero si una capa no aprende nada útil aún durante el entrenamiento, al menos la entrada seguirá avanzando por el modelo.

4.10.7 Capa de Normalización

La capa de normalización ("Layer Normalization" en inglés) ayuda acelerar el entrenamiento, mientras que estabiliza los valores evitando la explosión del gradiente o la desaparición forzada (como se ha visto anteriormente en las RNN's) y hace que las diferentes entradas del modelo permanezcan dentro de una misma escala. Esta se define de la siguiente forma

$$\mathbf{z}_i = \text{LayerNorm}(\mathbf{y}_i) = \gamma \cdot \frac{\mathbf{y}_i - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta \quad (132)$$

Donde, $\mu = \frac{1}{d} \sum_{j=1}^d y_{ij}$ es la media del vector $\mathbf{y}_i$, $\sigma^2 = \frac{1}{d} \sum_{j=1}^d (y_{ij} - \mu)^2$ es la varianza, $\gamma, \beta \in \mathbb{R}^{d_{model}}$ son parámetros aprendidos, y ϵ es un valor pequeño para prevenir la división por cero.

4.10.8 Capa Feed-Forward por posición

Esta capa se compone por dos transformaciones lineales con una activación ReLU en medio. Estas transformaciones se aplican de manera separada e idéntica a cada posición en la secuencia. De acuerdo con [28], esta operación se describe de la siguiente manera:

$$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2 \quad (133)$$

Donde $x \in \mathbb{R}^{d_{model}}$ es el vector de entrada, que hace referencia a una palabra que ya ha sido procesada por la capa de atención múltiple, más la suma de su posición codificada y además de estar normalizada, mientras que $W_1 \in \mathbb{R}^{512 \times 2048}$ y $W_2 \in \mathbb{R}^{2048 \times 512}$. Note que la dimensión intermedia $d_{ff} = 2048$ contribuye a que exista una expansión temporal de características, seguida de una compresión para regresar a la dimensión original.

En nuestro ejemplo siguiendo el proceso de una palabra, esto luce así

$$\mathbf{f}_i = \text{FFN}(\mathbf{z}_i) = \text{ReLU}(\mathbf{z}_i W_1 + b_1)W_2 + b_2 \quad (134)$$

4.10.9 Conexión Residual y Capa de Normalización

Una vez aplicada la capa Feed-Forward por posición, se realiza nuevamente una conexión residual, y se aplica normalización.

$$\mathbf{u}_i = \mathbf{z}_i + \mathbf{f}_i = \text{LayerNorm}(\mathbf{y}_i) + \text{ReLU}(\mathbf{z}_i W_1 + b_1)W_2 + b_2 \quad (135)$$

$$\mathbf{e}_{i_{enc}} = \text{LayerNorm}(\mathbf{u}_i) \quad (136)$$

Esto culmina el proceso del codificador aplicado a una sola palabra. En general, todos los cálculos se hacen con matrices, lo que facilita el proceso de entrenamiento ya que todo se hace en paralelo.

4.10.10 Sistema Completo - Codificador

Dada una secuencia de n palabras, cada una codificada como un vector “one-hot” de longitud V se tiene la matriz $X \in \mathbb{R}^{n \times V}$.

$$E = XW \quad \text{donde } W \in \mathbb{R}^{V \times d_{model}} \quad (137)$$

$$\tilde{E} = E + P \quad (P \text{ es la matriz de codificación posicional}) \quad (138)$$

$$\text{MHA}(\tilde{E}) = \text{Concat}(\text{Atención}_i(Q = \tilde{E}W_i^Q, K = \tilde{E}W_i^K, V = \tilde{E}W_i^V))W^O \quad (139)$$

$$Y = \tilde{E} + \text{MHA}(\tilde{E}) \quad (140)$$

$$Z = \text{LayerNorm}(Y) \quad (141)$$

$$F = \text{FFN}(Z) \quad (142)$$

$$U = Z + F \quad (143)$$

$$E_{enc} = \text{LayerNorm}(U) \quad (144)$$

4.10.11 Decodificador

El decodificador es la parte del modelo encargada de generar la secuencia de salida de manera autoregresiva, es decir, un elemento a la vez. Su estructura es semejante a la del codificador, pero con dos cambios importantes.

1. Enmascarada la atención, para evitar que el modelo acceda a tokens futuros durante el entrenamiento.
2. Se introduce el bloque de atención cruzada, el cual permite que el decodificador; consulte la salida del codificador.

4.10.12 Embeddings de entrada al decodificador

De forma análoga al codificador, el decodificador también transforma los tokens de salida generados hasta el momento en vectores numéricos mediante una matriz de embeddings. Para cada posición i , el token correspondiente se transforma en un vector $\mathbf{e}_i \in \mathbb{R}^{d_{model}}$, al cual se le suma una codificación posicional $\mathbf{p}_i$ para preservar el orden:

$$\tilde{\mathbf{e}}_i = \mathbf{e}_i + \mathbf{p}_i \quad (145)$$

Este vector $\tilde{\mathbf{e}}_i$ representa la entrada final al decodificador para la posición i , y se utiliza como punto de partida para los mecanismos de atención autoregresiva que siguen a continuación.

4.10.13 Atención enmascarada en el decodificador

Cuando se aplica atención en el codificador, cada posición puede atender a cualquier otra en la secuencia de entrada, incluso hacia adelante. Pero en el decodificador esto no se puede; de lo contrario, el modelo estaría haciendo trampa, ya que se estarían utilizando palabras futuras para predecir la actual. De manera

particular, como la entrada al decodificador es el vector $\tilde{\mathbf{e}}_i$, este se transforma en tres representaciones distintas.

$$Q_i = \tilde{\mathbf{e}}_i W^Q, \quad K_j = \tilde{\mathbf{e}}_j W^K, \quad V_j = \tilde{\mathbf{e}}_j W^V \quad (146)$$

donde $j \leq i$, ya que solo se puede atender a palabras anteriores o a sí misma. Luego se calcula la atención enmascarada para la posición i de la siguiente manera.

$$\text{Atención}_i = \sum_{j=1}^i \alpha_{i,j} V_j, \quad \text{donde } \alpha_{i,j} = \text{softmax}_j \left(\frac{Q_i \cdot K_j}{\sqrt{d_k}} \right) \quad (147)$$

Como ya se ha visto, en general se usan matrices para hacer el entrenamiento más rápido y de forma paralela, por lo que la operación matricial es análoga a la que se emplea en el codificador como muestra:

$$Q = \tilde{E}_{\text{dec}} W^Q, \quad K = \tilde{E}_{\text{dec}} W^K, \quad V = \tilde{E}_{\text{dec}} W^V$$

donde $\tilde{E}_{\text{dec}} \in \mathbb{R}^{n_{\text{dec}} \times d_{\text{model}}}$ es la secuencia completa de embeddings codificados posicionalmente. Por otro lado, la atención enmascarada se calcula de forma similar a la atención definida en el codificador, solo que en este caso se suma una matriz cuadrada M .

$$\text{Atención Enmascarada}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} + M \right) V \quad (148)$$

donde $M \in \mathbb{R}^{n \times n}$ es una máscara triangular inferior:

$$M_{i,j} = \begin{cases} 0 & \text{si } j \leq i \\ -\infty & \text{si } j > i \end{cases}$$

$$M = \begin{pmatrix} 0 & -\infty & -\infty & \dots & -\infty \\ 0 & 0 & -\infty & \dots & -\infty \\ 0 & 0 & 0 & \dots & -\infty \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & 0 \end{pmatrix}$$

De esta manera, al aplicar la función Softmax, los elementos enmascarados con $-\infty$ hacen que su probabilidad se vaya a 0, lo que garantiza que el modelo solo pueda usar información de tokens ya generados hasta el momento. Al igual que en el codificador, esta atención se implementa en múltiples cabezas paralelas y luego se combinan mediante una proyección lineal.

Se denotará a la salida individual por $\text{Masked-MHA}(\tilde{\mathbf{e}}_i)$, mientras que a la salida matricial por $\text{Masked-MHA}(\tilde{\mathbf{E}}_{\text{dec}})$

4.10.14 Conexión Residual y Normalización

Luego de aplicar la atención enmascarada, se vuelve a incorporar una conexión residual, para luego normalizar la suma del embedding del decodificador $\tilde{\mathbf{e}}_i$ y la salida de la atención enmascarada $\text{Masked-MHA}(\tilde{\mathbf{e}}_i)$.

$$\mathbf{y}_i = \tilde{\mathbf{e}}_i + \text{Masked-MHA}(\tilde{\mathbf{e}}_i) \quad (149)$$

$$\mathbf{z}_i = \text{LayerNorm}(\mathbf{y}_i) \quad (150)$$

4.10.15 Atención cruzada (Encoder–Decoder Attention)

En este punto, el decodificador contiene información generada de manera auto-regresiva por medio de $\mathbf{z}_i$, pero aún no incorpora ningún conocimiento acerca de la secuencia de entrada procesada por el codificador. Para integrar esta información, se introduce el mecanismo de atención cruzada. En esta etapa, la secuencia generada hasta el momento actúa como una consulta, mientras que las salidas del codificador funcionan como llaves y valores. El objetivo es permitir que el decodificador seleccione dinámicamente qué partes de la entrada codificada son más relevantes para generar cada token de salida.

Por lo tanto, la representación $\mathbf{z}_i$ se transforma en una consulta al ser multiplicada por la matriz W^Q como se muestra:

$$Q_i = \mathbf{z}_i W^Q \quad (151)$$

Mientras que las representaciones de la entrada, procesadas previamente en el codificador y almacenadas en la matriz E_{enc} , son proyectadas a llaves y valores:

$$K_j = E_{\text{enc},j} W^K, \quad V_j = E_{\text{enc},j} W^V \quad (152)$$

Por lo tanto, la atención cruzada para la posición i se calcula como:

$$\text{Atención}_i = \sum_{j=1}^{n_{\text{enc}}} \alpha_{i,j} V_j, \quad \text{donde } \alpha_{i,j} = \text{softmax}_j \left(\frac{Q_i \cdot K_j}{\sqrt{d_k}} \right) \quad (153)$$

Generalizando para toda la secuencia, se tienen las matrices:

$$Q = Z_{\text{dec}} W^Q, \quad K = E_{\text{enc}} W^K, \quad V = E_{\text{enc}} W^V \quad (154)$$

$$\text{Atención Cruzada}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V \quad (155)$$

Esta salida se suma de forma residual a la entrada del decodificador y se normaliza:

$$\mathbf{o}_{\text{dec}} = \text{LayerNorm}(Z_{\text{dec}} + \text{Atención Cruzada}(Q, K, V)) \quad (156)$$

Esto permite que el modelo combine la información autoregresiva generada en el decodificador con la información codificada de la secuencia de entrada, guiando de forma efectiva el proceso de generación.

4.10.16 Capa Feed-Forward por posición

Véase que $\mathbf{o}_{\text{dec}}$ contiene tanto la información autoregresiva generada hasta ese momento, como la información relevante de la secuencia de entrada. Por lo que el siguiente paso consiste en transformar esta representación mediante una red neuronal completamente conectada, justo como ocurrió en el codificador.

$$\text{FFN}(\mathbf{o}_i) = \text{ReLU}(\mathbf{o}_i W_1 + b_1) W_2 + b_2 \quad (157)$$

Finalmente, como es costumbre tras aplicar atención, se aplica una conexión residual y una normalización por capa.

$$\mathbf{u}_i = \mathbf{o}_i + \text{FFN}(\mathbf{o}_i) \quad (158)$$

$$\mathbf{h}_i = \text{LayerNorm}(\mathbf{u}_i) \quad (159)$$

De forma matricial

$$\mathbf{H}_{\text{dec}} = \text{LayerNorm}(\mathbf{O}_{\text{dec}} + \text{FFN}(\mathbf{O}_{\text{dec}})) \quad (160)$$

donde $\mathbf{O}_{\text{dec}} \in \mathbb{R}^{n_{\text{dec}} \times d_{\text{model}}}$ es la matriz que contiene las representaciones finales de cada posición tras aplicar atención cruzada.

4.10.17 Predicción final y generación autoregresiva

La salida del decodificador luego de atravesar todo el sistema es una matriz $\mathbf{H}_{\text{dec}} \in \mathbb{R}^{n_{\text{dec}} \times d_{\text{model}}}$, donde cada fila representa la codificación final de una posición en la secuencia generada. El objetivo principal ahora es convertir estas representaciones en predicciones de tokens del vocabulario V definido desde el inicio.

Para ello, se aplica una proyección lineal que transforma cada vector de dimensión d_{model} en un vector de puntuaciones ("logits") de dimensión V , el tamaño del vocabulario:

$$\text{logits}_t = \mathbf{h}_t W^{\text{out}} + b^{\text{out}}, \quad \text{con } W^{\text{out}} \in \mathbb{R}^{d_{\text{model}} \times V} \quad (161)$$

Luego, se aplica la función softmax para convertir estas puntuaciones en una distribución de probabilidad sobre todos los tokens posibles:

$$\mathbf{p}_t = \text{softmax}(\text{logits}_t) \in \mathbb{R}^V \quad (162)$$

El token con mayor probabilidad es seleccionado como predicción para la posición t . Este proceso se repite paso a paso de manera autoregresiva: en cada iteración, el token generado se concatena a la secuencia de entrada al decodificador, permitiendo que se prediga el siguiente token. Este ciclo continúa hasta que se alcanza un token especial de fin de secuencia $\langle \text{EOS} \rangle$ o se supera una longitud máxima.

Durante la fase de entrenamiento, este proceso se acelera mediante el uso de la secuencia de salida completa en paralelo, enmascarando las posiciones futuras con una máscara triangular inferior. En cambio, durante la inferencia, el modelo genera la secuencia **token por token**.

4.11 D³VAE

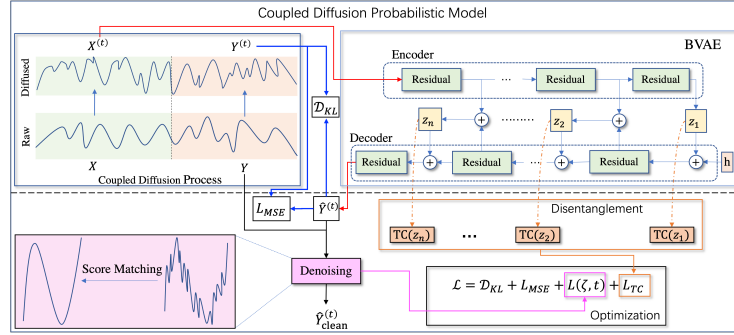


Figura 14: Estructura de DVAE, imagen de [15]

El modelo D³VAE busca modelar la distribución de la serie temporal observada x como una función de variables latentes z , las cuales se asumen provenientes de una distribución gaussiana multivariada. A diferencia de los modelos clásicos, D³VAE introduce un proceso de difusión acoplado con técnicas de denoising score matching y una estrategia de disentanglement en el espacio latente. Esto permite una generación más robusta y estructurada de series temporales, manteniendo consistencia temporal y separando distintos patrones dinámicos presentes en los datos (tendencia, estacionalidad, etc.).

4.11.1 Formulación del Problema

Considérese una serie temporal multivariada $X = \{x_1, \dots, x_n\}$, donde cada $x_i \in \mathbb{R}^d$ representa un conjunto de d variables predictoras, incluyendo la variable de respuesta, y sea $Y = \{y_{n+1}, \dots, y_{n+m}\}$ la variable de respuesta que se desea pronosticar. Se asume la existencia de una variable latente $Z \sim p(Z|X)$ desde la cual se puede generar la distribución de la serie objetivo mediante una función paramétrica $f(\cdot)$. De esta forma, la distribución de Y queda definida como...

$$p(\mathbf{Y}) = \int p(\mathbf{Z}|\mathbf{X}) \cdot p(\mathbf{Y}|f(\mathbf{Z})), d\mathbf{Z} \quad (163)$$

Por lo tanto, el objetivo principal del modelo es aprender una representación latente Z que capture las señales importantes de X que permitan reconstruir de forma precisa la serie futura Y .

4.11.2 Proposición 1: Descomposición de señal y ruido

Sea X una serie temporal multivariada de entrada, y sea X_r su componente ideal, es decir, libre de ruido. Suponga que toda serie temporal puede descomponerse como...

$$X = X_r + \epsilon_X \quad (164)$$

Donde ϵ_X representa el ruido inherente a las observaciones, y se asume que $X_r \perp \epsilon_X$, es decir, son estadísticamente independientes.

Bajo esta suposición, la distribución de la variable Z , condicionada a X , puede expresar como...

$$p(Z|X) = p(Z|X_r, \epsilon_X) \quad (165)$$

Del mismo modo, la serie objetivo Y puede descomponerse como

$$Y = Y_r + \epsilon_Y \quad (166)$$

donde Y_r es la parte ideal de la variable de respuesta, y ϵ_Y representa el ruido en la salida. (tal como la aleatoriedad, errores de medición, o factores modelados).

El modelo se entrena para capturar completamente Y_r , es decir, se asume que puede modelarse sin error.

$$\hat{Y}_r \approx Y_r \quad (167)$$

Entonces, el error total entre la predicción y la variable de respuesta es

$$\|\hat{\mathbf{Y}} - \mathbf{Y}\| = \|\hat{\mathbf{Y}}_r + \epsilon_{\hat{\mathbf{Y}}} - (\mathbf{Y}_r + \epsilon_Y)\| = \|\epsilon_{\hat{\mathbf{Y}}} - \epsilon_Y\| > 0 \quad (168)$$

lo cual puede deberse a la incertidumbre en el proceso de generación o a la limitante del modelo y la falta de información.

4.11.3 Proceso de Difusión Acoplada

D³VAE implementa un proceso de difusión acoplada, en el que tanto los datos de entrada X como la serie objetivo Y se degradan progresivamente mediante la incorporación de ruido gaussiano a través de T pasos. El proceso de difusión directa se define de la siguiente forma...

$$q(X^{(1:T)}, X^{(0)}) = \prod_{t=1}^T q(X^{(t)}|X^{(t-1)}) \quad (169)$$

Donde

$$q(X^{(t)}|X^{(t-1)}) = \mathcal{N}(X^{(t)}; \sqrt{1 - \beta_t}X^{(t-1)}, \beta_t I) \quad (170)$$

En esta formulación, β_t controla la cantidad de ruido agregada en cada paso t , la cual proviene de una serie creciente $\beta = \{\beta_1, \dots, \beta_T \mid \beta_t \in [0, 1]\}$. Por otra parte $X^{(0)}$ representa los datos originales de la serie de tiempo. Al aumentar t , la señal se vuelve progresivamente más ruidosa hasta aproximarse a una distribución gaussiana estándar en el paso final T .

Definiendo $\alpha_t = 1 - \beta_t$ y $\bar{\alpha} = \prod_{s=1}^t \alpha_s$, se puede reescribir la distribución como una función directa de los datos originales.

$$q(\mathbf{X}^{(t)}|\mathbf{X}^{(0)}) = \mathcal{N}(\mathbf{X}^{(t)}; \sqrt{\bar{\alpha}_t}\mathbf{X}^{(0)}, (1 - \bar{\alpha}_t)I) \quad (171)$$

Recordando la preposición 1, $X^{(0)}$ se puede descomponer en $X_r + \epsilon_X$, por lo tanto la muestra $X^{(t)}$ queda de la siguiente manera

$$X^{(t)} = \sqrt{\bar{\alpha}_t}X^{(0)} + (1 - \bar{\alpha}_t)\delta_X := \langle \sqrt{\bar{\alpha}_t}X_r, \sqrt{\bar{\alpha}_t}\epsilon_X + (1 - \bar{\alpha}_t)\delta_X \rangle \quad (172)$$

donde el término $\delta_X = \frac{\epsilon}{\sqrt{1-\bar{\alpha}_t}}$, con $\epsilon \sim \mathcal{N}(0, I)$. Ahora, estableciendo a $\tilde{X}_r^{(t)} = \sqrt{\bar{\alpha}_t}X_r$ y $\delta_{\tilde{X}}^{(t)} = \sqrt{\bar{\alpha}_t}\epsilon_X + (1 - \bar{\alpha}_t)\delta_X$, dada la preposición 1 y la ecuación 121, se tiene lo siguiente...

$$p_\phi(Z^{(t)}|X^{(t)}) = p_\phi(Z^{(t)}|\tilde{X}_r^{(t)}, \delta_{\tilde{X}}^{(t)}), \quad p_\theta(\hat{Y}^{(t)}|Z^{(t)}) = p_\theta(\hat{Y}_r^{(t)}|Z^{(t)})p_\theta(\delta_{\hat{Y}}^{(t)}|Z^{(t)}), \quad (173)$$

donde claramente $\delta_{\hat{Y}}^{(t)}$ denota el ruido generado por $\hat{Y}^{(t)}$.

Una vez definido el proceso de difusión sobre la serie temporal X , se aplica un procedimiento análogo a la serie objetivo $Y = Y^{(0)} \sim q(Y^{(0)})$. Esto permite modelar y mitigar la incertidumbre aleatoria inherente a los datos.

$$\begin{aligned} Y^{(t)} &= \sqrt{\bar{\alpha}'_t}Y^{(0)} + (1 - \bar{\alpha}'_t)\delta_Y = \sqrt{\bar{\alpha}'_t}Y_r + \sqrt{\bar{\alpha}'_t}\epsilon_Y + (1 - \bar{\alpha}'_t)\delta_Y \\ &= \langle \tilde{Y}_r^{(t)}, \delta_{\tilde{Y}}^{(t)} \rangle \end{aligned} \quad (174)$$

Donde $\omega \in (0, 1)$, y se define a $\beta'_t = \omega \cdot \beta_t$, $\rightarrow \alpha'_t = 1 - \beta'_t$ y $\bar{\alpha}'_t = \prod_{s=1}^t \alpha'_s$. Finalmente, tanto la serie temporal de entrada X como la variable de respuesta Y son enriquecidas y suavizadas con ruido controlado, lo cual permite mejorar la capacidad de generalización del modelo, en particular en series temporales cortas o anómalas.

Se pueden establecer dos resultados importantes (Ver el apéndice B de [15]).

1. Lema 1. Existe un modelo probabilístico f tal que la parte ideal de la serie objetivo Y puede aproximarse a partir de la entrada difusa.

$$\hat{Y}_r^{(t)} = f_{\phi, \theta}(X^{(t)}), \quad \text{y} \quad \mathcal{D}_{KL}(q(\tilde{Y}_r^{(t)})||p(\hat{Y}_r^{(t)})) < \epsilon \quad (175)$$

2. Lema 2. El proceso de difusión acoplada permite reducir la brecha entre el ruido generado por el modelo y el ruido inherente de los datos.

$$\lim_{t \rightarrow \infty} \mathcal{D}_{KL}(q(\delta_{\tilde{Y}}^{(t)})||p_\theta(\delta_{\tilde{Y}}^{(t)}|Z^{(t)})) < \mathcal{D}_{KL}(q(\epsilon_Y)||p_\theta(\epsilon_{\hat{Y}})) \quad (176)$$

Esto trae como consecuencia, que la incertidumbre creada por el modelo generativo y el ruido inherente de los datos pueda ser reducido por medio del proceso de difusión acoplada. Además, ayuda a mejorar la capacidad de generalización del modelo ya que ocurre un incremento de los datos tanto de la serie de entrada como de la serie objetivo.

4.11.4 Auto-codificador Variacional Bidireccional (BVAE)

A diferencia de los Auto-codificadores de variación unidireccional (VAE), el auto-codificador Variacional bidireccional (BVAE) usa una sola red neuronal parametrizada para codificar y decodificar. Es decir, usa los mismo pesos para inferir Y dado X , y para predecir X dado Y . Las VAE's usan una arquitectura para codificar y otra para decodificar. Como consecuencia, esto implican un mayor costo computacional al tener el doble de parámetros que la BVAE.

En general, se tiene como objetivo encontrar la mejor distribución de los datos $p(X|\theta)$ a partir de muestras $\{X^{(n)}\}_{n=1}^N$, si X depende de variables observables Z y θ hace referencia a los parámetros del sistema. Esto nos lleva a...

$$p(x|\theta) = \mathbb{E}_{z|\theta}[p(x|z, \theta)] = \int_z p(x|z, \theta)p(z, \theta)dz. \quad (177)$$

Esto implica hacer demasiados cálculos, lo cual no es posible en la práctica, para ello se crea la solución al proponer el VAE [12], pero BVAE encuentra una solución mucho más elegante al proponer un flujo bidireccional donde se aproxima la distribución $q_f(z|x, \theta)$ por medio del flujo hacia adelante $q_{\text{hacia adelante}}$ de la red neuronal, esto conforma al codificador, mientras que el decodificador por medio del flujo inverso en la red neuronal $p_{\text{en retroceso}}$ aproxima la distribución original $p_b(x|z, \theta)$ por medio del vector generado $z = \mu(x) + \sigma(x) \cdot \epsilon$ tal que $\epsilon \in \mathcal{N}(0, I)$ [13]. De esta manera se puede generar una muestra de datos X a partir de ruido gaussiano, y este principio es aplicado dentro del modelo D³VAE al generar el vector $Z = \{z_1, \dots, z_n\}$ tal que $z_i \in \mathbb{R}^m$ y $z_i = [z_{i,1}, \dots, z_{i,m}]$ a partir de la serie de tiempo con ruido gaussiano $X^{(t)}$, donde cada $z_{i+1} \sim p(z_{i+1}|z_i, X)$. Luego, n se determina en base a la cantidad de bloques residuales en el codificador/decodificador. De esta forma el decodificador obtiene la distribución $p_\theta(\hat{Y}^{(t)}|Z^{(t)}) = p_\theta(\hat{Y}_r|Z^{(t)})p_\theta(\delta_{\hat{Y}}^{(t)}|Z^{(t)})$

4.11.5 Corrección de series generadas y Scaled Denoising Score Matching

El propósito de Scaled Denoising Score Matching (SDSM) es limpiar la serie temporal generada $p_\theta(\hat{Y}^{(t)})$, que fue generada a partir de $X^{(t)}$ en el proceso BVAE entrenado para minimizar la divergencia de Kullback-Leibler entre la distribución difusa y la distribución generada.

Por lo tanto, una vez obtenida la predicción $\hat{Y}^{(t)}$, esta se compara directamente con la serie difusa $Y^{(t)}$ mediante una pérdida de error cuadrático medio L_{MSE} y una penalización $\mathcal{D}_{KL}$, y es precisamente esto lo que causa que la distribución de $\hat{Y}^{(t)}$ se acerca a la distribución difusa corrupta $Y^{(t)}$. Aun que en principio este proceso ayuda a modelar la distribución de los datos, introduce ruido que aleja las muestras generadas de los valores verdaderos. Para contrarrestar este efecto, entra SDSM.

SDSM entrena una función de energía $E(\hat{Y}; \zeta)$ para estimar el score de la distribución $\hat{Y}$, lo que permite aplicar un paso de actualización que ayuda a limpiar la predicción. Por lo tanto, los autores [15] establecen el objetivo...

$$L_{\text{DSM}}(\zeta) = \mathbb{E}_{p_{\sigma_0}(\hat{Y}, Y)} \|\nabla_{\hat{Y}} \log(q_{\sigma_0}(\hat{Y}|Y)) + \nabla_{\hat{Y}} E(\hat{Y}; \zeta)\|^2, \quad (178)$$

Donde $p_{\sigma_0}(\hat{Y}|Y) = \frac{1}{(2\pi\sigma_0^2)^{d/2}} \exp(-\frac{\|\hat{Y}-Y\|^2}{2\sigma_0^2})$ es la unión de la densidad de los pares corruptos y limpios de las muestras $(\hat{Y}, Y)$. $\nabla_{\hat{Y}} \log(q_{\sigma_0}(\hat{Y}|Y))$ es la derivada de la densidad logarítmica de un solo kernel de ruido (σ_0), en el caso particular del ruido gaussiano, se tiene que $\log(q_{\sigma_0}(\hat{Y}|Y)) = \frac{-(\hat{Y}-Y)^2}{2\sigma_0^2} + C$, el

cual está dedicado a remplazar el estimador de la densidad de Parzen: $p_\sigma(\hat{Y}) = \int q_{\sigma_0}(\hat{Y}|Y)p(Y)dY$ en el score matching.

$E(\hat{Y}, \zeta)$ es una función de energía, la cual es una red neuronal parametrizada por ζ (los autores no definen la forma de E , pero en la practica puede ser una red de 2–3 capas completamente conectadas o convolucionales), donde su entrada es $\hat{Y}$, y su salida es un escalar $E(\hat{Y}) \in \mathbb{R}$. Cabe destacar que, aun que la función de energía regresa un escalar, su gradiente $\nabla_{\hat{Y}}E(\hat{Y}; \zeta)$ es un vector que apunta en la dirección en la que se debe mover $\hat{Y}$ para reducir su energía. Al hacer la sustitución en la ecuación 127.

$$L_{\text{DSM}}(\zeta) = \mathbb{E}_{p_{\sigma_0}(\hat{Y}, Y)} \left\| -\frac{1}{\sigma_0^2}(\hat{Y} - Y) + \nabla_{\hat{Y}}E(\hat{Y}; \zeta) \right\| \quad (179)$$

$$= \mathbb{E}_{p_{\sigma_0}(\hat{Y}, Y)} \left\| -\frac{\sigma_0^2}{\sigma_0^2}(\hat{Y} - Y) + \sigma_0^2 \nabla_{\hat{Y}}E(\hat{Y}; \zeta) \right\| \quad (180)$$

$$= \mathbb{E}_{p_{\sigma_0}(\hat{Y}, Y)} \|Y - \hat{Y} + \sigma_0^2 \nabla_{\hat{Y}}E(\hat{Y}; \zeta)\|^2 \quad (181)$$

Por lo tanto, para la serie ruidosa $\hat{Y}^{(t)}$ en el paso t , se tiene...

$$L_{\text{DSM}}(\zeta, t) = \mathbb{E}_{p_{\sigma_0}(\hat{Y}^{(t)}, Y)} \|Y - \hat{Y}^{(t)} + \sigma_0^2 \nabla_{\hat{Y}^{(t)}}E(\hat{Y}^{(t)}; \zeta)\|^2 \quad (182)$$

Para escalar el ruido en diferentes niveles, los autores [15] emplean una serie monótona decreciente con valores σ fijos $\{\sigma_1, \dots, \sigma_T \mid \sigma_t = 1 - \bar{\sigma}_t\}$, donde cada $\bar{\sigma}_t = \prod_{s=1}^t \alpha_s$. Finalmente, el objetivo del Multi-Escalado DSM es

$$L(\zeta, t) = \mathbb{E}_{p_{\sigma_0}(\hat{Y}^{(t)}|Y)p(Y)} \|Y - \hat{Y}^{(t)} + \sigma_0^2 \nabla_{\hat{Y}^{(t)}}E(\hat{Y}^{(t)}; \zeta)\|^2 \quad (183)$$

Donde $\sigma \in \{\sigma_1, \dots, \sigma_T\}$ y $l(\sigma_t) = \sigma_t$, de esta forma el gradiente tendrá la magnitud correcta al establecer σ_0 , ya que sino hubiese ruido ($\sigma_0 \rightarrow 0$), el gradiente se volvería infinito.

Durante inferencia, en el pronóstico de series temporales generativas, las muestras generadas $\hat{Y}$ se prueban sin aplicar el proceso de difusión. Por otro lado, para quitar más ruido a la serie temporal generada $\hat{Y}$, se aplica un salto de reducción de ruido a través de un paso de gradiente.

$$\hat{Y}_{\text{limpia}} = \hat{Y} - \sigma_0^2 \nabla_{\hat{Y}}E(\hat{Y}; \zeta) \quad (184)$$

Los autores encontraron, que los resultados tienden a poseer una distribución espacial mucho más grande que la variable de respuesta Y , y el termino de ruido en la ecuación anterior aproxima el ruido generado entre la serie generada y la variable de respuesta "limpia". Por lo tanto, $\sigma_0^2 \nabla_{\hat{Y}}E(\hat{Y}; \zeta)$ puede entenderse como la incertidumbre estimada de la predicción. [15]

4.11.6 Desentrelazando Z para Interpretabilidad

La interpretabilidad del modelo de series temporales generativas es esencial para diversas aplicaciones prácticas. Con el fin de mejorar la interpretabilidad y la

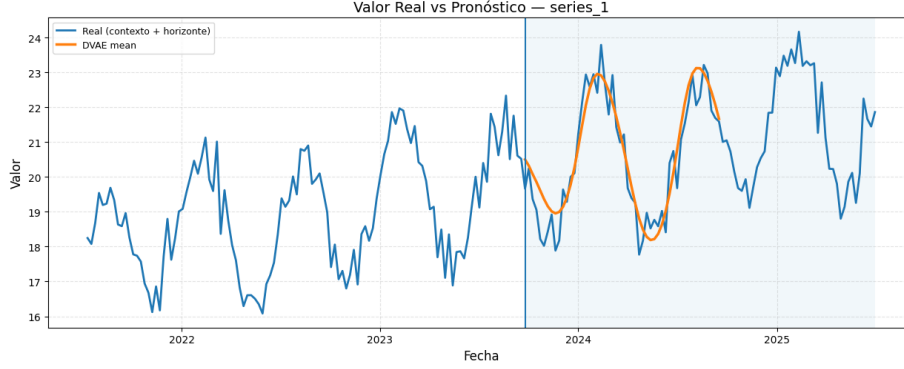


Figura 15: Valor real y pronóstico generado por el modelo D³VAE para una serie temporal sintética.

fiabilidad de las predicciones, se busca desentrelazar las variables latentes generadas por el BVAE. Este desentrelazado se realiza minimizando la correlación total (TC), una medida que cuantifica el grado de dependencia entre múltiples variables aleatorias. Por lo tanto, para cada variable latente z_i , los autores [15] definen la Correlación Total (TC) de z_i como...

$$\text{TC}(z_i) = \mathcal{D}_{KL}(p_\phi(z_i) || \bar{p}_\phi(z_i)), \quad \bar{p}_\phi(z_i) = \prod_{j=1}^m p_\phi(z_{i,j}) \quad (185)$$

Donde $p_\phi(z_i)$ representa la distribución conjunta real de los elementos de z_i , mientras que $\bar{p}_\phi(z_i)$ representa una distribución que asume la independencia entre los elementos $z_{i,j}$. Recuérdese que cada $z_i = [z_{i,1}, \dots, z_{i,m}]$, por lo que m denota el número de dimensiones del vector latente z_i , que además necesitan ser desentrelazados.

Minimizar la $\text{TC}(z_i)$ representa acercar la distribución conjunta a una donde los elementos sean independientes, lo que ayuda a desentrelazar las variables latentes. Los autores mencionan que obtener una TC muy baja puede implicar que las variables latentes no capturen información útil, pero dado que en BVAE cada z_i es construido a partir del codificador y decodificador (Figura 14), se obtienen representaciones enriquecidas, lo que mitiga este riesgo. Por otro lado, para combatir el efecto potencial de valores irregulares, los autores calculan la pérdida de correlación total de BVAE al promediar las correlaciones totales de $z_{1:n}$ [15].

$$L_{\text{TC}} = \frac{1}{n} \sum_{i=1}^n \text{TC}(z_i) \quad (186)$$

4.11.7 Entrenamiento y Predicción

Para reducir el efecto de incertidumbre, los autores [15] proponen un proceso de difusión acoplada equipado con una red neuronal de reducción de ruido, sin tener que sacrificar la capacidad de generalización del modelo. Posteriormente, se realiza el desentrelazado de las variables latentes Z al minimizar la correlación total (TC), como se discutió previamente. Por último, los autores reconstruyeron la función de pérdida global utilizando todas las contribuciones previamente definidas, como se muestra...

$$\mathcal{L} = \psi \cdot \mathcal{D}_{KL}(q(Y^{(t)}) || p_{\theta}(\hat{Y}^{(t)})) + \lambda \cdot L(\zeta, t) + \gamma \cdot L_{TC} + L_{\text{MSE}}(\hat{Y}^{(t)}, Y^{(t)}) \quad (187)$$

Donde, L_{MSE} calcula el error cuadrado entre $\hat{Y}^{(t)}$ y $Y^{(t)}$, $L(\zeta, t)$ corresponde a la pérdida de Scaled Denoising Score Matching (SDSM) y L_{TC} corresponde a la pérdida asociada al desentrelazado de las variables latentes. De esta manera el modelo generativo aprende al minimizar la función objetivo (187).

5 Metodología

Esta sección describe el pipeline metodológico implementado para pronosticar la inflación en México (INPC) y evaluar el efecto de distintas familias de variables exógenas y modelos. El enfoque compara tres configuraciones de entrada y dos protocolos de validación, utilizando la **optimización bayesiana** (BO) con **MAE** como métrica objetivo.

5.1 Conjunto de datos y frecuencia

El Índice Nacional de Precios al Consumidor (INPC) se obtiene del portal del INEGI en frecuencia quincenal y mensual. Si bien la frecuencia mensual es la empleada oficialmente y la más habitual en la literatura económica, en esta investigación se adopta finalmente una **frecuencia semanal**, obtenida mediante una interpolación lineal a partir de los datos quincenales.

Esta decisión se fundamenta en que la **estructura fundamental del proceso inflacionario se mantiene** tras la desagregación temporal: las tendencias, ciclos y estacionalidades observadas en la versión mensual permanecen intactas, sin introducir ruido estructural significativo. La granularidad adicional de la frecuencia semanal permite capturar variaciones de corto plazo y dinámicas transitorias que podrían pasar desapercibidas a nivel mensual. En modelos neuronales, esta mayor densidad temporal mejora la sensibilidad del aprendizaje a cambios abruptos y a la estacionalidad de alta frecuencia, sin alterar la coherencia macroeconómica del fenómeno subyacente. En consecuencia, se conserva la esencia del proceso inflacionario, pero se proporciona a los algoritmos una secuencia temporal más informativa y detallada.

De forma general, el **framework** desarrollado admite distintas configuraciones de frecuencia temporal según las características del conjunto de datos o el dominio de aplicación. El usuario puede especificar la frecuencia mediante el parámetro **freq**, el cual acepta las siguientes opciones:

- "W-Mon": frecuencia **semanal**, con semanas iniciando en lunes (configuración por defecto utilizada en este estudio);
- "ME": frecuencia **mensual**, que toma el último día de cada mes;
- "B": frecuencia de **días hábiles** (*Business Days*), útil para series financieras o de alta frecuencia.

Esta flexibilidad permite adaptar el sistema tanto a contextos macroeconómicos (como la inflación) como a dominios de ventas, producción o finanzas, manteniendo la consistencia de la estructura experimental.

5.2 Preprocesamiento y transformaciones

Sea y_t la serie de interés (inflación). En el experimento original, para estabilizar la dinámica y favorecer el aprendizaje en modelos neuronales se aplica **diferencia de orden 1** sobre la tasa de cambio (Δy_t), seguida de estandarización y

reescalado. Todas las transformaciones se realizan con **anti-fuga**: sus parámetros se calculan exclusivamente con datos previos a cada ventana de validación.

El pipeline extendido descompone primero $y_t = \tau_t + c_t$. Las transformaciones **diff**, **diff_logp1**, **pct**, **logp1**, **none** y **diff2** se permiten únicamente sobre τ_t y se invierten antes de reconstruir el pronóstico. El ciclo c_t no se diferencia ni se transforma mediante logaritmos, debido a que puede tomar valores negativos; sólo se estandariza con la media y desviación del entrenamiento. Las variables de calendario y las componentes FFT se calculan por separado para cada componente y a partir de su respectiva historia.

5.3 Ingeniería de variables

Se estudian tres escenarios de covariables:

- A. **Target únicamente**: el modelo observa solo y_t .
- B. **Target + temporales cíclicas**: se añaden codificaciones seno/coseno de *mes*, *trimestre* y *día del mes* para capturar estacionalidad calendario.
- C. **Target + temporales cíclicas + FFT**: además de lo anterior, se incorporan **señales senoidales** derivadas de la *Transformada Discreta de Fourier* aplicada al target. Se seleccionan las top- k componentes (aquí $k = 8$) por energía, que se **extrapolan** al horizonte futuro para modelar periodicidades persistentes.

5.4 Modelos

Se evaluaron: **Holt-Winters**, **XGBoost**, **RNN**, **LSTM**, **DeepAR** y **Transformer** (Vanilla). Cada modelo respeta la misma política de transformaciones y covariables por escenario. Para RNN, LSTM, DeepAR y Transformer se agregó una implementación propia en PyTorch con una interfaz común. RNN y LSTM emplean un codificador recurrente y una cabeza directa multihorizonte; DeepAR utiliza una LSTM autorregresiva con verosimilitud gaussiana; Transformer utiliza atención causal, codificación posicional y una cabeza multihorizonte. NeuralForecast se conserva únicamente para reproducir los experimentos históricos.

En el pipeline HP, cada arquitectura se instancia dos veces. Ambos modelos comparten el horizonte y el protocolo de validación, pero ajustan parámetros y escalas independientes para τ_t y c_t . El resultado evaluado no es cada componente de forma aislada, sino la serie reconstruida $\hat{y}_t = \hat{\tau}_t + \hat{c}_t$.

5.5 Protocolos de validación

Se emplean dos esquemas complementarios:

- **Validación simple (baseline)**: una partición con fecha de corte fija (*single split*). Útil como línea base y para depuración.

- **Validación cruzada temporal (rolling-origin)**: se definen W ventanas **no solapadas** de **12 meses** (o tamaño elegido por el usuario), donde en cada ventana se re-entrena y evalúa el mismo modelo. Este protocolo reduce el sesgo por selección de una única partición y mide estabilidad en el tiempo.

La **métrica principal** es el **MAE**, por su interpretabilidad en niveles y su menor sensibilidad a valores atípicos en este caso de estudio.

5.6 Optimización de hiperparámetros

Para cada modelo y escenario de variables se utiliza **Optimización Bayesiana** con función objetivo:

$$\theta^* = \arg \min_{\theta} \text{MAE}_{\text{val}}(\theta),$$

donde θ parametriza la arquitectura (p. ej., número de capas/neuronas en RN-N/LSTM; profundidad, *learning rate* y rondas en XGBoost; cabezas/anchos en Transformer; estacionalidad en Holt-Winters). La BO balancea exploración/explotación mediante una función de adquisición (p. ej., *Expected Improvement*) y permite encontrar configuraciones competitivas con pocas evaluaciones.

5.7 Protocolo de pronóstico final

Una vez seleccionada θ^* en cada ventana, el modelo se **re-entrena con toda la información disponible** previa al horizonte y se generan las predicciones. En el esquema *single split*, para mantener consistencia con la validación, el pronóstico se produce desplazando $-m$ meses según la ventana de validación definida por el usuario.

5.8 Controles de sobreajuste y linealidad

Además de la validación cruzada, se implementa un chequeo de **linealidad de pronósticos**: para una trayectoria $\hat{y}_t$ (horizonte), se ajusta por mínimos cuadrados la recta $mt + b$ y se calcula el $\text{RMSE}(\hat{y}_t, mt + b)$. Si el RMSE cae por debajo de un umbral, la predicción se considera esencialmente lineal y se descarta como no informativa respecto a una regresión o promedio históricos. Este control evita aceptar configuraciones que *aprenden* soluciones triviales.

5.9 Diseño del experimento

Se comparan los tres escenarios (A–C) en los horizontes de 6 y 12 meses, bajo ambos protocolos (baseline y rolling-origin). Para cada combinación (modelo, escenario, ventana) se realizan N iteraciones de BO (p. ej., 20–30), reportando el **promedio** y la **dispersión** del MAE en validación cruzada. El mejor método se selecciona por MAE promedio y estabilidad (desviación estándar) a lo largo de las ventanas.

La extensión HP conserva estos horizontes y fechas de corte. Dentro de cada ventana se ejecutan, en orden, la descomposición restringida al entrenamiento, el ajuste de transformaciones y escalas, el entrenamiento independiente de los dos componentes, su inversión y la reconstrucción final. De esta forma, la comparación directa y la comparación HP utilizan exactamente los mismos valores reales y la misma métrica.

5.10 Arquitectura del framework y reproducibilidad

El sistema se implementa en Python con módulos desacoplados:

- **Preprocesamiento** (`process_data.py`, `utiles.py`): limpieza INEGI, remuestreo, transformaciones, codificación temporal y generación de seno-*idales* (FFT).
- **Funciones objetivo** (`obj_bayes.py` y variantes `cv`): definen la pérdida (MAE), horizontes, frecuencias y *anti-leakage*.
- **Ajuste** (`fit_bayes_opt.py` y `cross_v*`): BO por modelo/escenario y por ventana (rolling).
- **Predicción** (`predict_bayes.py`): re-entrena con θ^* y produce $\hat{y}$ fuera de muestra.
- **Orquestación** (`orquestador.py`, `main.py`): define `cutoff`, horizonte, métrica, frecuencia, iteraciones y ejecuta `train_n_predict()`; guarda métricas y predicciones.

El Anexo A presenta un ejemplo de la búsqueda bayesiana utilizada en el pipeline histórico. La extensión se encuentra aislada en el paquete `inpc_forecasting`. Sus módulos contienen la descomposición HP, transformaciones invertibles, ingeniería de variables por componente, modelos PyTorch y una interfaz de línea de comandos gobernada por YAML. Se fija la semilla 119, se habilita ejecución en CPU o GPU, se aplica detención temprana y recorte del gradiente, y se guarda un identificador de la configuración en cada archivo de resultados. Los módulos heredados con sufijo `cv` continúan implementando el protocolo histórico de validación cruzada temporal.

6 Resultados y discusión

Esta sección resume los hallazgos experimentales del estudio comparativo. Se evaluaron dos protocolos: (i) *single split* como línea base y (ii) *validación cruzada temporal* (*rolling-origin*) con ventanas de 12 meses. La métrica principal es el **MAE** debido a su interpretabilidad en niveles y a que, en este caso de estudio, resulta menos sensible a perturbaciones puntuales. Los resultados se reportan para horizontes de 6 y 12 meses.

6.1 Diseño de evaluación

Cada evaluación fija una fecha de corte, re-entrena el modelo con la información disponible hasta dicho corte y evalúa el desempeño en el horizonte futuro. En el protocolo de validación cruzada, este procedimiento se repite sobre múltiples ventanas temporales, lo que permite estimar el MAE promedio y su dispersión. En consecuencia, la CV reduce el riesgo de conclusiones dependientes de una única partición temporal y ofrece una estimación más estable del desempeño fuera de muestra.

6.2 Escenarios de variables

Se compararon tres configuraciones:

- A. **Target:** sólo y_t .
- B. **Target + temporales cíclicas:** codificaciones sen/cos de *mes*, *trimestre* y *día del mes*.
- C. **Target + temporales cíclicas + FFT:** además, **top- k** senoidales (en este trabajo $k = 8$) obtenidas con DFT del target y extrapoladas al horizonte.

6.3 Línea base vs. validación cruzada

La Tabla 2 compara el MAE (media $\pm$ sd) bajo *single split* y validación cruzada. El patrón general es consistente: el *single split* tiende a producir estimaciones más optimistas o más volátiles, mientras que la validación cruzada estabiliza la medición al promediar múltiples cortes temporales. En particular, la dispersión del error entre ventanas permite identificar modelos con desempeño robusto, en lugar de modelos favorecidos por un único periodo de evaluación.

Tabla 2: MAE (media $\pm$ sd) por protocolo y horizonte.

Modelo	6 meses		12 meses	
	Single split	CV (12m)	Single split	CV (12m)
Naive	$0,5 \pm 0,28$	$0,51 \pm 0,27$	$0,58 \pm 0,27$	$0,65 \pm 0,29$
Promedio	$0,45 \pm 0,24$	$0,6 \pm 0,18$	$0,5 \pm 0,18$	$0,72 \pm 0,09$
RWD	$0,5 \pm 0,29$	$0,53 \pm 0,17$	$0,59 \pm 0,29$	$0,6 \pm 0,2$
Holt-Winters	$1,6 \pm 0,95$	$0,97 \pm 1,02$	$1,07 \pm 0$	$1,22 \pm 1,4$
XGBoost	$0,79 \pm 0,7$	$0,84 \pm 0,35$	$1,08 \pm 0,84$	$0,76 \pm 0,14$
RNN	$0,53 \pm 0,24$	$0,5 \pm 0,3$	$0,66 \pm 0,4$	$0,64 \pm 0,38$
LSTM	$0,56 \pm 0,38$	$0,51 \pm 0,32$	$0,68 \pm 0,44$	$0,65 \pm 0,35$
DeepAR	$0,54 \pm 0,4$	$0,49 \pm 0,22$	$0,56 \pm 0,34$	$0,79 \pm 0,37$
Transformer	$0,65 \pm 0,32$	$0,8 \pm 0,37$	$0,82 \pm 0,42$	$0,45 \pm 0,18$
D ³ VAE	$0,54 \pm 0,24$	$0,44 \pm 0,23$	$0,48 \pm 0,11$	$0,43 \pm 0,14$

6.4 Impacto de las variables exógenas

La Tabla 3 reporta el MAE promedio en CV bajo los escenarios A/B/C. En términos generales, incorporar variables temporales cíclicas tiende a mejorar el desempeño respecto a usar únicamente el target, mientras que las senoidales FFT aportan valor adicional cuando la serie presenta periodicidad marcada en el horizonte de evaluación. Sin embargo, el efecto no es universal: en modelos altamente expresivos, añadir exógenas puede aumentar la varianza o inducir sobreajuste dependiendo del protocolo y el horizonte.

Tabla 3: MAE promedio (CV, 12m) por modelo y escenario de variables.

Modelo	A) Target	B) Target+Temp	C) +FFT (k=8)
Naive	$0,65 \pm 0,29$	—	—
Promedio	$0,72 \pm 0,09$	—	—
RWD	$0,6 \pm 0,2$	—	—
Holt-Winters	$1,22 \pm 1,4$	—	—
XGBoost	$0,76 \pm 0,14$	$0,80 \pm 0,57$	$0,87 \pm 0,14$
RNN	$0,64 \pm 0,38$	$0,77 \pm 0,35$	$0,88 \pm 0,77$
LSTM	$0,65 \pm 0,35$	$0,64 \pm 0,33$	$0,74 \pm 0,47$
DeepAR	$0,79 \pm 0,37$	$0,84 \pm 0,34$	$0,67 \pm 0,58$
Transformer	$0,45 \pm 0,18$	$0,76 \pm 0,29$	$0,72 \pm 0,32$
D ³ VAE	$0,43 \pm 0,14$	$0,58 \pm 0,26$	$0,59 \pm 0,37$

6.5 Ranking de modelos y estabilidad

La Figura 16 sintetiza el ranking bajo validación cruzada, mostrando MAE promedio y dispersión (sd). El criterio de selección final prioriza simultáneamente *bajo MAE y baja variabilidad*. Bajo este criterio, D³VAE presenta el mejor desempeño y estabilidad global en ambos horizontes, mientras que Transformer destaca en 12 meses pero con mayor variación en 6 meses.

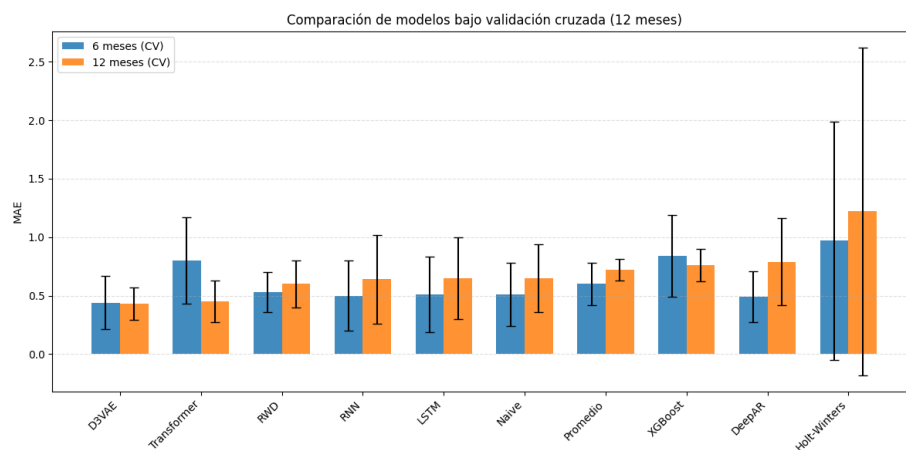


Figura 16: Comparación del MAE bajo validación cruzada (12 meses) para horizontes de 6 y 12 meses. Las barras representan el promedio y las líneas verticales la desviación estándar.

6.6 Casos ilustrativos

Para complementar el análisis agregado, se presentan ejemplos de pronóstico frente a valores reales, con el fin de visualizar patrones de desempeño y fallas típicas (e.g., predicciones lineales, sobreajuste al conjunto de validación). Las Figuras 17 y 18 muestran configuraciones representativas del comportamiento de LSTM bajo distintas combinaciones de variables y suavizados.

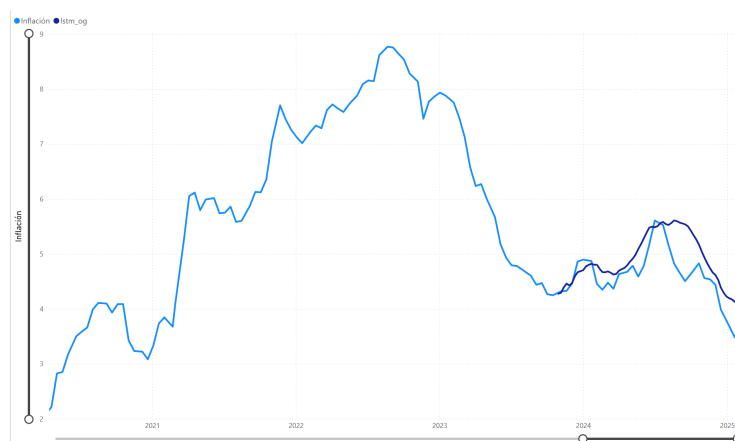


Figura 17: Ejemplo ilustrativo de pronóstico con LSTM bajo una configuración de hiperparámetros optimizada.

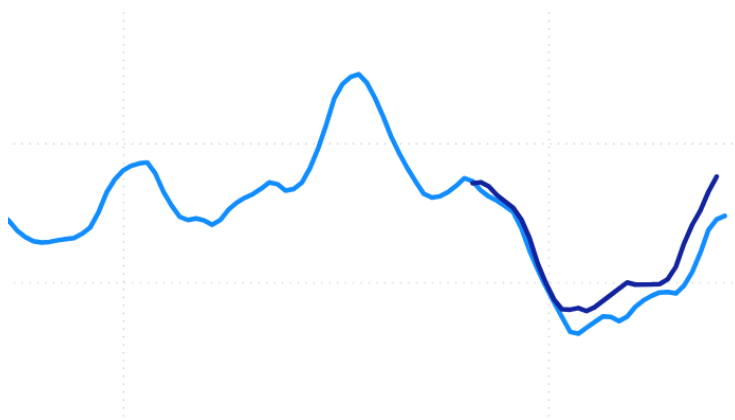


Figura 18: Ejemplo ilustrativo de pronóstico con LSTM incorporando suavizado exponencial para reducir ruido de alta frecuencia sin alterar la estructura general de la señal.

6.7 Control de linealidad

Durante la optimización, se observó que ciertas configuraciones de redes neuronales (en particular con arquitecturas excesivamente grandes) convergen a soluciones casi lineales en el horizonte, lo cual constituye un comportamiento degenerado. Para detectar y penalizar estas soluciones, se ajusta por mínimos cuadrados la recta $mt + b$ a $\hat{y}_t$ y se evalúa el error $\text{RMSE}(\hat{y}_t, mt + b)$. Si el RMSE cae por debajo de un umbral, se aplica una penalización fija al valor objetivo, forzando a la optimización bayesiana a explorar configuraciones con dinámica no trivial. Este mecanismo evita aceptar configuraciones que no superen a una regresión o a un promedio histórico.

6.8 Discusión frente a la literatura

Los resultados confirman que la complejidad de una arquitectura no garantiza por sí misma un menor error. Este comportamiento es consistente con la importancia de la estacionalidad señalada por [5]: cuando la representación temporal no coincide con el horizonte o introduce redundancia, una red con mayor capacidad puede aumentar su varianza en lugar de mejorar su generalización. Del mismo modo, el desempeño competitivo de las líneas base obliga a interpretar las diferencias junto con su dispersión y no únicamente a partir del mejor valor observado en una ventana.

El mejor comportamiento de Transformer a doce meses es compatible con su capacidad para relacionar posiciones distantes mediante atención, mientras que la variabilidad de RNN y LSTM coincide con sus limitaciones para conservar dependencias largas, discutidas por [25]. DeepAR aporta una formulación probabilística útil, pero su ventaja teórica no implica automáticamente una mejora en MAE cuando se dispone de una sola serie agregada; el modelo fue conce-

bido para aprovechar conjuntos de series relacionadas [24]. Por otra parte, el desempeño de D³VAE sugiere que representar incertidumbre y regularizar el espacio latente puede ser valioso, aunque esta conclusión permanece limitada a los cortes y horizontes evaluados.

6.9 Extensión HP y resultados pendientes

La implementación descrita en esta versión deja preparado el experimento que pronostica τ_t y c_t de manera independiente con RNN, LSTM, DeepAR y Transformer propios. Sin embargo, una corrida mínima de verificación no es metodológicamente comparable con el benchmark completo. Por esta razón, no se sustituyen las métricas anteriores ni se afirma todavía que la descomposición HP reduzca el error. La hipótesis deberá evaluarse mediante todas las ventanas rolling-origin de seis y doce meses.

Tabla 4: Matriz de resultados prevista para el benchmark HP. Los valores permanecen pendientes hasta ejecutar el protocolo completo.

Modelo	MAE 6 meses	sd 6 meses	MAE 12 meses	sd 12 meses
RNN	Pendiente	Pendiente	Pendiente	Pendiente
LSTM	Pendiente	Pendiente	Pendiente	Pendiente
DeepAR	Pendiente	Pendiente	Pendiente	Pendiente
Transformer	Pendiente	Pendiente	Pendiente	Pendiente

6.10 Hallazgos principales

En conjunto, los resultados sustentan cuatro hallazgos principales:

- La **validación cruzada temporal** produce estimaciones más estables y representativas que el *single split*, reduciendo la dispersión del MAE entre ventanas.
- Las **transformaciones** para inducir estacionariedad fueron determinantes para evitar comportamientos triviales y estabilizar el entrenamiento, especialmente en modelos neuronales.
- Las **variables temporales cíclicas** no mejoran el desempeño respecto al escenario A en la mayoría de los modelos. Su efecto depende del horizonte y de la arquitectura: en algunos casos la red aprende información útil de estas covariables, pero el beneficio no se mantiene de manera consistente entre ventanas.
- En el ranking global bajo CV, **D³VAE** presenta la mejor combinación de desempeño y estabilidad, mientras que **Transformer** destaca particularmente en el horizonte de 12 meses.

7 Conclusiones

En este trabajo se abordó el pronóstico de la inflación en México mediante un estudio comparativo entre modelos clásicos, enfoques basados en árboles y arquitecturas neuronales (incluyendo modelos probabilísticos y generativos). Los resultados muestran que, aunque los modelos simples ofrecen líneas base competitivas, su desempeño y estabilidad dependen fuertemente del protocolo de evaluación. En particular, la validación cruzada temporal fue esencial para obtener conclusiones robustas y evitar selecciones guiadas por un único periodo histórico.

En relación con los objetivos planteados, se construyó el conjunto semanal del INPC, se estableció un protocolo común de comparación, se evaluaron transformaciones y variables temporales, y se integraron implementaciones propias de RNN, LSTM, DeepAR y Transformer. También quedó implementada la descomposición HP sin fuga temporal y el entrenamiento independiente de tendencia y ciclo. No obstante, la hipótesis específica sobre una reducción del MAE mediante esta descomposición permanece abierta: aceptarla o rechazarla requiere ejecutar el benchmark HP completo y no puede inferirse de las pruebas mínimas de funcionamiento.

A lo largo de la experimentación se observó que **la parsimonia** juega un papel central. Reducir espacios paramétricos, limitar la complejidad de arquitecturas y controlar soluciones degeneradas (como pronósticos casi lineales) mejoró la capacidad de generalización. Este hallazgo es consistente con la práctica recomendada en el diseño de redes neuronales: comenzar con modelos simples y aumentar complejidad únicamente cuando exista evidencia de mejora fuera de muestra.

En términos de variables, se encontró que las **codificaciones temporales cíclicas** aportan información útil en la mayoría de escenarios, mientras que las **senoidales FFT** pueden contribuir cuando la periodicidad es marcada, especialmente en horizontes más largos, aunque su beneficio no es universal y la mayor parte de los experimentos lejos de tener un beneficio introdujo más ruido a los modelos.

Finalmente, bajo el criterio conjunto de *bajo MAE* y *baja variabilidad* en validación cruzada, el modelo **D³VAE** presentó el mejor desempeño global, seguido por **Transformer** como alternativa competitiva en el horizonte de 12 meses. Estos resultados sugieren que los enfoques generativos probabilísticos constituyen una vía prometedora para el pronóstico inflacionario cuando se requiere modelar incertidumbre y obtener predicciones robustas a diferentes ventanas temporales, mientras que el mejor desempeño del modelo Transformer puede explicarse por su capacidad para modelar dependencias temporales de largo alcance de manera explícita. A diferencia de arquitecturas recurrentes (RNN, LSTM) y de modelos autoregresivos probabilísticos como DeepAR, el Transformer no comprime la información histórica en un estado oculto secuencial, sino que accede directamente a todo el contexto de observaciones mediante mecanismos de atención.

Como líneas de trabajo futuro, resulta natural extender este estudio en varias direcciones. En primer lugar, considerar el modelado de múltiples componentes

del índice inflacionario permitiría analizar de forma más detallada la contribución de cada rubro al proceso agregado, así como evaluar si ciertos modelos capturan mejor dinámicas sectoriales específicas.

En segundo lugar, resulta relevante estudiar periodos caracterizados por cambios estructurales más pronunciados, como episodios de alta inflación, choques externos o modificaciones en la política monetaria, con el fin de evaluar la robustez de los modelos ante rupturas en el proceso generador de datos.

Asimismo, los resultados sugieren la necesidad de profundizar en estrategias que permitan incorporar de manera más efectiva la información espectral obtenida mediante la descomposición de Fourier. Si bien las senoidales dominantes aportan información estacional relevante, se observó que su inclusión no siempre se traduce en una mejora consistente en la generalización. Explorar mecanismos que permitan seleccionar, ponderar o regularizar estas señales de forma adaptativa podría conducir a modelos más estables.

Un hallazgo particularmente interesante es que, en ciertos casos, arquitecturas como LSTM logran modelar el proceso inflacionario con alta precisión cuando la estacionalidad es capturada adecuadamente. Esto sugiere que el desafío no radica únicamente en la capacidad del modelo, sino en lograr que la red neuronal aprenda la estructura estacional de forma efectiva y generalizable, sin sobreajustarse a patrones específicos del conjunto de entrenamiento y validación.

Finalmente, se observó de manera consistente que la inclusión directa de variables temporales explícitas (codificaciones de calendario) no mejoró el desempeño predictivo y, en muchos casos, lo deterioró. Este comportamiento sugiere que dichas variables pueden introducir redundancia o ruido cuando el modelo ya es capaz de inferir relaciones temporales a partir de la propia serie y de representaciones espectrales. Este resultado refuerza la importancia de evaluar críticamente la utilidad de las variables exógenas y de evitar su incorporación automática sin un análisis previo de su impacto en la generalización.

A Código de optimización para RNN

Listing 1: Búsqueda Bayesiana para RNN

```
from bayes_opt import BayesianOptimization
from functools import partial
import pandas as pd

def fit_rnn(data, cutoff_date, iteraciones, freak, Metric,
            horizon, Mes_val, feats, transf, signals):
    rnn_params = {
        "years": (5, 12),
        "months": (Mes_val, Mes_val),
        "h": (horizon, horizon),
        "input_size": (6, 9),
        "neurons": (3, 4),
```



```

        "layers": (1, 2),
        "max_steps": (10, 70),
        "freq": (freq_map[freak], freq_map[freak]),
        "metric": (metric_map[Metric], metric_map[Metric]),
        "feats": (feats, feats),
        "signals": (signals, signals),
        "transf": (transformaciones_map[transf],
                    transformaciones_map[transf]),
    }

    data = data.assign(ds=pd.to_datetime(data["ds"]))
    data = data[data["ds"] <= cutoff_date]

    rnn_objective = partial(obj_bayes.obj_rnn, data=data)

    opt = BayesianOptimization(
        f=rnn_objective,
        pbounds=rnn_params,
        random_state=119,
        verbose=2,
        allow_duplicate_points=False,
    )
    opt.maximize(init_points=3, n_iter=iteraciones)

    best = dict(opt.max["params"])
    best_score = opt.max["target"]    # p.ej., -MAE (mayor es
                                     mejor)
    return best, best_score

```

Referencias

- [1] Mohit Apte and Yashodhara Haribhakta. Advancing financial forecasting: A comparative analysis of neural forecasting models n-hits and n-beats. *arXiv preprint arXiv:2409.00480*, 2024.
- [2] Jeannine Bailliu, Daniel Garcés, Mark Kruger, and Miguel Messmacher. Explaining and forecasting inflation in emerging markets: the case of mexico. Technical report, Bank of Canada, 2003.
- [3] bbva. ¿qué es la inflación y cómo se calcula? <https://www.bbva.es/finanzas-vistazo/ef/finanzas-personales/que-es-la-inflacion-y-como-se-calcula.html>, 2018. Accedido el 15 de Febrero de 2025.
- [4] Christopher M Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.

- [5] Carlos Capistrán and M Ramos-Francia. Uso de modelos estacionales para pronosticar la inflación a corto plazo en México. *Documentos de Investigación*, 5, 2009.
- [6] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, pages 785–794, 2016.
- [7] Robert J Hodrick and Edward C Prescott. Postwar us business cycles: an empirical investigation. *Journal of Money, Credit and Banking*, 29(1):1–16, 1997.
- [8] Rob J Hyndman and George Athanasopoulos. *Forecasting: principles and practice*. OTexts, 2018.
- [9] Rob J Hyndman and George Athanasopoulos. *Forecasting: Principles and Practice*. OTexts, 3rd edition, 2021.
- [10] Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani. *An Introduction to Statistical Learning with Applications in R*. Springer, 2021.
- [11] Diederik P. Kingma and Max Welling. Auto-encoding variational bayes. In *International Conference on Learning Representations*, 2014.
- [12] Diederik P Kingma, Max Welling, et al. Auto-encoding variational bayes, 2013.
- [13] Bart Kosko and Olaoluwa Adigun. Bidirectional variational autoencoders. In *2024 International Joint Conference on Neural Networks (IJCNN)*, pages 1–9. IEEE, 2024.
- [14] Solomon Kullback and Richard A Leibler. On information and sufficiency. *The Annals of Mathematical Statistics*, 22(1):79–86, 1951.
- [15] Yan Li, Xinjiang Lu, Yaqing Wang, and Dejing Dou. Generative time series forecasting with diffusion, denoise, and disentanglement. In Alice H. Oh, Alekh Agarwal, Danielle Belgrave, and Kyunghyun Cho, editors, *Advances in Neural Information Processing Systems*, 2022.
- [16] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2013.
- [17] Kevin P Murphy. *Probabilistic Machine Learning: Advanced Topics*. MIT Press, 2023.
- [18] Emi Nakamura. Inflation forecasting using a neural network. *Economics Letters*, 86(3):373–378, 2005.

- [19] National Institute of Standards and Technology. 6.4.3.2 forecasting with single exponential smoothing. <https://www.itl.nist.gov/div898/handbook/pmc/section4/pmc432.htm#Single%20Exponential%20Smoothing%20with>, 2012. Accedido el 9 de mayo de 2025.
- [20] National Institute of Standards and Technology. 6.4.3.3 forecasting with double exponential smoothing. <https://www.itl.nist.gov/div898/handbook/pmc/section4/pmc433.htm>, 2012. Accedido el 9 de mayo de 2025.
- [21] Morten O. Ravn and Harald Uhlig. On adjusting the hodrick–prescott filter for the frequency of observations. *The Review of Economics and Statistics*, 84(2):371–376, 2002.
- [22] Stephen J. Roberts. Lecture 7: Spectral analysis. <https://www.robots.ox.ac.uk/~sjrob/Teaching/SP/17.pdf>, 2019. Lecture notes for Statistical Pattern Recognition, University of Oxford.
- [23] Héctor Pedroza Robles. Uso de redes neuronales para el pronóstico de la inflación en México use of neural networks for inflation forecasting in Mexico. *Almanaque Económico*, page 62, 2019.
- [24] David Salinas, Valentin Flunkert, Jan Gasthaus, and Tim Januschowski. DeepAR: Probabilistic forecasting with autoregressive recurrent networks. *International journal of forecasting*, 36(3):1181–1191, 2020.
- [25] Alex Sherstinsky. Fundamentals of recurrent neural network (rnn) and long short-term memory (lstm) network. *Physica D: Nonlinear Phenomena*, 404:132306, 2020.
- [26] Robert H Shumway and David S Stoffer. *Time series analysis and its applications: with R examples*. Springer, 2006.
- [27] Gregory Trubetskoy. Triple exponential smoothing (forecasting). <https://grisha.org/blog/2016/01/29/triple-exponential-smoothing-forecasting/>, 2016. Accedido el 9 de mayo de 2025.
- [28] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.